

CSE6234-SOFTWARE DESIGN

ASSIGNMENT:

E-Commerce system

Project Name:

VantageMarket

Group: TT5L_G2

Name	ID
LEONG WEI LEE	243UC247G7
HAREEN A/L VIJAYA KUMAR	242UC244Q0
LAI WEI JIAN	1211109103
YEE YUNG SIEN	243UC247DS

Introduction.....	4
1.1. Abstract	4
1.2. Problem Statement	4
1.3. Project Objectives	5
1.4. Literature Review.....	6
1.5 Project and System Scope	7
1.5.1 Project Scope	7
1.5.2 System Scope	7
In-Scope Subsystems and Functionalities:	7
Out-of-Scope Features:	7
2. System Overview	8
2.1 Generic Use Case diagram.....	8
2.2. Primary Actors.....	9
Actor 1: Guest/Registered customer.....	10
2.2.1 Browse Catalogue.....	10
2.2.2 Search Products.....	12
2.2.3 Manage Shopping Cart.....	14
• 2.2.4 Login	16
2.2.5 Register as a registered user	18
2.2.6 Process Checkout	20
2.2.7 Track Order	22
Actor 2 Administrator	24
2.3.1 Manage Inventory	24
2.3.2 Manage Promotions.....	26
2.3.3 Generate Reports	28
2.4 Class Diagram.....	30
2.5 ERD	31
2.5 Sequence diagram.....	32
2.5.1 Browse catalogue	32
2.5.2 Search products	33
2.5.3 Manage Shopping Cart.....	33
2.5.4 Register User	34
2.5.5 Process Checkout (Applying Strategy Pattern).....	35
2.5.6 Track Order.....	36

2.5.7 Manage Inventory(Admin)	36
2.5.8 Manage Promotions(Admin).....	37
2.5.9 Generate Reports (Admin)	38
2.6 Object diagram.....	39
3. Requirements	40
Software Design Concepts and Design Principles.....	40
3.1 The SOLID Principles	40
3.1.1 S-Single responsibility principle	40
3.1.2 O - Open/Closed Principle (OCP):.....	44
3.1.3 L - Liskov Substitution Principle (LSP):.....	47
3.1.4 I - Interface Segregation Principle (ISP):.....	51
3.1.5 D - Dependency Inversion Principle (DIP):	55
4. Proposed Design Patterns	60
4.1 Design Pattern 1: The Strategy Pattern	60
4.1.1 Description of workflow or data flow	60
4.1.2 Sample Class Diagram	60
4.1.3 Sample Other UML Notations (Sequence Diagram)	61
4.1.4 Sample Potential Code	62
4.1.5. Benefit.....	64
4.1.6. Limitations.....	64
4.2 Design Pattern 2: The Observer Pattern.....	65
4.2.1. Function or Software Component Affected	65
4.2.2. Description of workflow or data flow.....	65
4.2.3. Sample Class Diagram.....	65
4.2.4 Sample Other UML Notations (Sequence Diagram)	67
4.2.5. Sample Potential Code.....	67
4.2.6. Benefit.....	70
4.2.7. Limitations	70
4.3 Design Pattern 3: The Singleton Pattern	71
4.3.1. Description of workflow or data flow.....	71
4.3.2 Sample Class Diagram.....	72
4.3.4 Sample Potential Code	75
4.3.5. Benefit.....	76
4.3.6. Limitations	77
5. Conclusion and suggestion	78
6. Bibliography/Reference	79

Introduction

1.1. Abstract

This project aims to develop a web-based e-commerce system inspired by modern online shopping platforms such as Amazon. The system is designed using React for the frontend, Python and PHP for backend functionalities, and incorporates multiple software design patterns to improve maintainability, scalability, and code reusability. The primary focus of this project is the application of at least three software design patterns within the system architecture to demonstrate their effectiveness in solving common software design problems. The developed system includes essential e-commerce functionalities such as user registration and login, product browsing, shopping cart management, order placement, and order tracking. Design patterns such as Model-View-Controller (MVC), Singleton, and Observer are implemented to enhance separation of concerns, resource management, and event-driven notifications respectively. Through the application of these patterns, the project aims to create a modular and extensible software system that can be easily maintained and expanded in future development stages.

This project demonstrates how software design patterns can be effectively integrated into real-world web applications, particularly in e-commerce systems where system organization and maintainability are critical.

1.2. Problem Statement

As general e-commerce platforms scale to accommodate millions of concurrent users and expansive product catalog, traditional, tightly coupled software architectures struggle to maintain performance, stability, and security. During peak traffic events (such as flash sales or holiday promotions), monolithic system designs often face severe bottlenecks, where a failure in one minor component—such as the review system or a specific payment gateway—can cause the entire checkout process to crash.

Furthermore, hardcoding business logic limits a platform's flexibility. Integrating new payment methods, updating dynamic pricing rules, or managing real-time inventory syncing across concurrent user sessions becomes highly complex and prone to errors. Therefore, there is a critical need to investigate and apply established software design patterns and modular architectural principles to decouple system components. This ensures high functional independence, allowing the platform to scale efficiently, secure user data through proper encapsulation, and remain highly available without requiring complete system rewrites.

1.3. Project Objectives

The primary goal of this study is to evaluate and design a software e-commerce platform. This will be achieved through the following specific objectives:

1. **To establish a modular system architecture:** To evaluate and apply a decoupled architectural model (such as a Three-Tier Layered Architecture) that strictly separates the presentation interface, business logic, and database layers to improve system maintainability and scalability.
2. **To resolve high-coupling in checkout processes:** To investigate the application of the **Strategy Pattern** in decoupling external payment gateways from the core shopping cart module, allowing the system to seamlessly interchange payment algorithms (e.g., Credit Card, E-Wallets) without altering internal code.
3. **To ensure real-time data consistency:** To analyse the use of the **Observer Pattern** for asynchronous event handling, ensuring that inventory stock changes automatically and reliably push updates to user Wishlist's and active shopping carts.
4. **To maximize functional independence:** To define clear boundaries and encapsulation protocols for core modules (User Management, Product Catalogue, and Order Processing) so that each module maintains high cohesion and can be independently refactored or updated without disrupting the overall user experience.

1.4. Literature Review

The usability of e-commerce platforms has long been recognized as a critical factor in customer satisfaction and retention. Psaromiligkos (2003) introduced the DEPTH framework, which integrates design patterns with heuristic evaluation to identify recurring usability issues such as navigation clarity, checkout flows, and product search. This approach demonstrated that design patterns serve as reusable solutions to interface challenges, and that structured evaluation methods can reduce support costs while improving overall user experience (Psaromiligkos, 2003).

Scalability and resilience in e-commerce systems have increasingly relied on microservices-based architectures. The International Journal of Scientific Development and Research (2025) emphasized the importance of patterns such as API Gateway, Service Discovery, and Event-Driven Architecture in supporting distributed systems. The study highlighted containerization and load balancing as key enablers of horizontal scaling, with case studies showing how these approaches strengthen inventory, payment, and authentication services. This research underscores the necessity of modular, loosely coupled architectures for handling high transaction volumes while maintaining performance and reliability (International Journal of Scientific Development and Research, 2025).

Beyond usability and scalability, ecosystem sustainability depends on standardization principles. Prajapati (2022) conducted a systematic literature review and multi-case study to identify six meta-requirements and nineteen design principles for e-commerce ecosystems. By analyzing platforms such as Amazon, Airbnb, and Uber, the study demonstrated how boundary resources like APIs and SDKs foster developer participation and multi-homing. Standardization was shown to amplify scalability, network effects, and interoperability, ultimately strengthening the ecosystem's long-term sustainability (Prajapati, 2022).

Overall, these three studies provide a comprehensive foundation for designing modern e-commerce platforms. Usability frameworks ensure customer-centric design (Psaromiligkos, 2003), microservices architectures enable scalability and resilience (International Journal of Scientific Development and Research, 2025), and ecosystem standardization principles foster interoperability and sustainability (Prajapati, 2022). Synthesizing these perspectives highlights the importance of integrating design patterns and modular architectures to create e-commerce systems that are not only technically robust but also user-friendly, scalable, and adaptable to future growth.

1.5 Project and System Scope

1.5.1 Project Scope

The scope of this project is to design and develop a web-based e-commerce system inspired by modern online shopping platforms such as Amazon. The system will be built utilizing React for the frontend, alongside Python and PHP for backend functionalities. A primary focus of the project is the practical application of established software design patterns—specifically Model-View-Controller (MVC), Singleton, Observer, and Strategy patterns—to resolve common architectural bottlenecks such as high coupling and resource management. The project aims to establish a modular, Three-Tier Layered Architecture that strictly separates the presentation interface, business logic, and database layers to ensure system maintainability and scalability. The final deliverable will be a well-structured software architecture capable of handling essential transaction workflows. Hardware deployment, physical network configuration, and business marketing plans are strictly out of scope for this assignment.

1.5.2 System Scope

The system scope defines the functional boundaries of the platform. The system is designed to interact with three primary human actors (Guest Users, Registered Customers, and Administrators) and external system actors (Payment Gateways and Courier APIs).

In-Scope Subsystems and Functionalities:

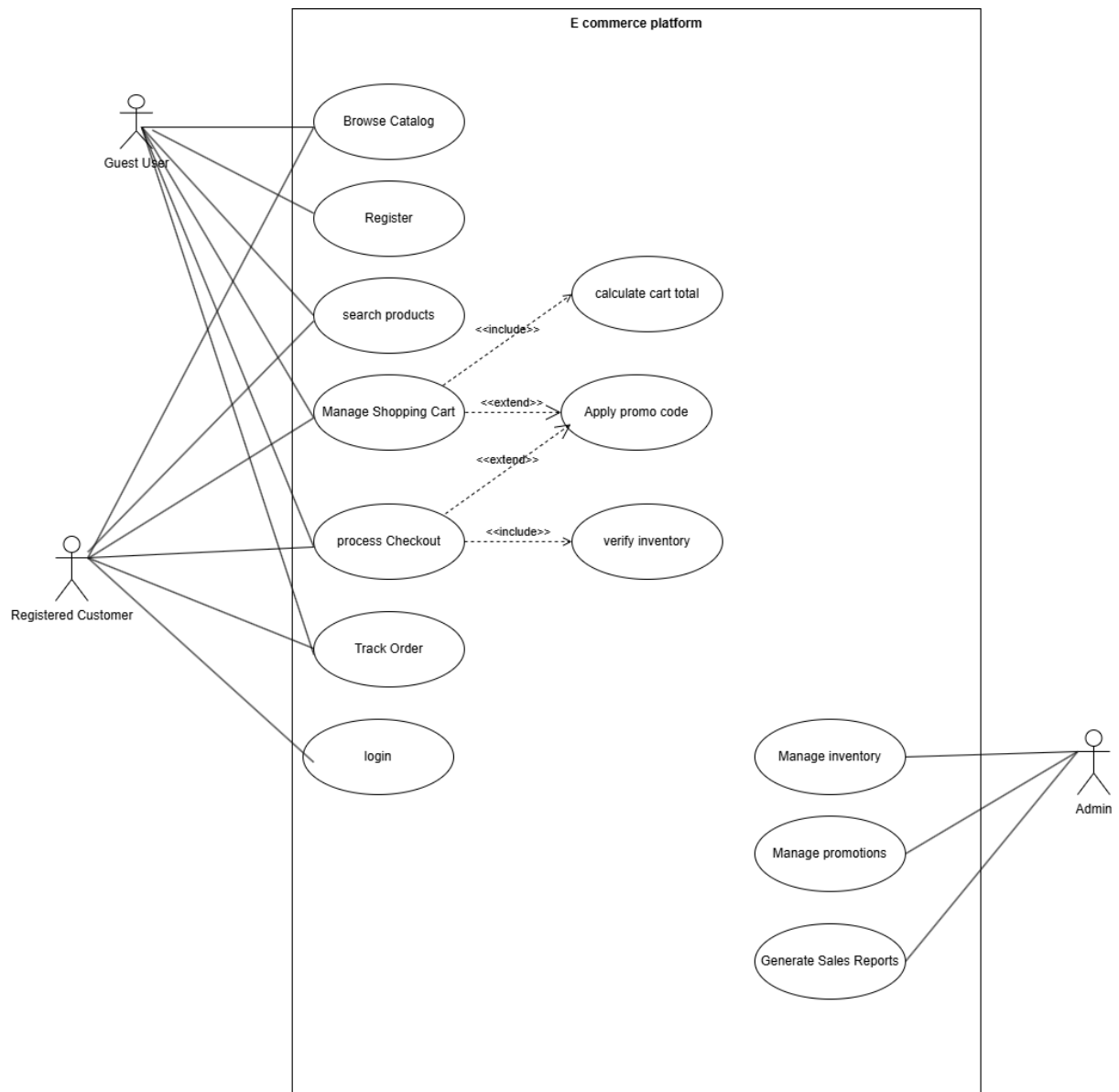
- **User Management:** Functionality for guest users to create permanent accounts, secure login authentication, and user profile management.
- **Product Catalogue and Search:** Features allowing users to browse product categories, apply specific filters (e.g., price range, brand), and execute keyword-based searches against the database.
- **Shopping Cart Module:** Temporary session management allowing users to add, remove, and update item quantities while dynamically calculating subtotals and estimated taxes.
- **Checkout and Payment Processing:** Finalizing orders by entering shipping details and seamlessly routing transactions to external Payment Gateways, utilizing the Strategy Pattern for flexible algorithm interchange.
- **Order Tracking:** Providing post-purchase visibility by requesting and formatting live location data from external Courier APIs.
- **Inventory Management:** Secure administrative tools to add products, update stock quantities, and remove items. This includes utilizing the Observer Pattern to ensure real-time data consistency by automatically notifying active shopping carts when an item goes out of stock.

Out-of-Scope Features: To maintain high cohesion and ensure functional independence among the core modules, the following real-world features are excluded from the current system design:

- **Third-Party Seller Portals:** The platform assumes a single-retailer model; marketplace features for independent vendors are excluded.
- **Fulfilment Logistics:** Internal warehouse routing, barcode scanning, and physical delivery driver applications are out of scope.
- **Advanced AI Recommendations:** Machine-learning-driven personalized recommendation engines based on user behaviour are not included.

2. System Overview

2.1 Generic Use Case diagram



2.2. Primary Actors

Actors are the entities (human or external systems) that interact directly with your platform.

Guest User	• Browse product catalogue • Search products by keyword and filters • Manage shopping cart • Register for an account • Process checkout • Track order
Registered user	• Browse product catalogue • Search products by keyword and filters • Manage shopping cart • Register for an account • Process checkout • Track order • Log in into account • Receive stock notifications
Administrator	• Manage inventory • Manage promotions • Generate reports

Actor 1: Guest/Registered customer

2.2.1 Browse Catalogue

As a guest, I want to browse through various product categories so that I can explore the items available for sale without having to log in first.

Use case ID	UC01
Primary Actor:	Guest user/registered user
Brief Description:	This use case describes the process of a user navigating the e-commerce platform's catalogue, applying specific search filters, and viewing detailed information about individual products.
Trigger	The user accesses the e-commerce platform and clicks on a product category or the main catalogue navigation link.
Relationships	Association: Guest User, Registered Customer Include: None Extend: None Generalization: None
Preconditions	The database containing product information is operational.
Post conditions	• Success: The user successfully views the catalogue or a specific product's details. • Failure: The user receives an error message if products cannot be loaded.
Normal Flow of events	1. The user navigates to the main catalogue or selects a specific product category from the navigation menu. 2. The system retrieves the relevant products from the database. 3. The system displays the products in a grid/list view along with available filtering options (e.g., Price Range, Brand, Rating). 4. The user selects one or more filters to narrow down the search results. 5. The system dynamically updates the displayed product list to match the user's selected criteria. 6. The user clicks on a specific product thumbnail or title from the list. 7. The system retrieves and displays the detailed product page (including high-resolution images, full descriptions, pricing, and stock availability).
Alternative Flow	• 1a. Empty Category: If the selected category has no active products, the system displays a "No Products Found in this Category" message with a button to

	return to the main catalogue.
Exceptions	Database Unreachable: The system fails to load the product catalogue due to a database connection timeout. The system displays a "Temporary Maintenance" error page to the user.
Business rules	Out-of-Stock Logic: Products with an inventory count of zero should still appear in the catalog for SEO and Wishlist purposes, but they must be visually tagged with an "Out of Stock" badge and pushed to the bottom of sorted search results.

2.2.2 Search Products

As a guest, I want to use a search bar to find specific items by keywords or filters so that I can quickly locate the products I am interested in.

Use case ID	UC02
Primary Actor:	Guest user/registered user
Brief Description:	This use case describes the process of a user entering text queries into a search bar, and how the system processes that input to retrieve and display matching products from the catalog.
Trigger	The user clicks on the search bar and begins typing a keyword or phrase.
Relationships	Association: Guest User, Registered Customer Include: None Extend: None Generalization: None
Preconditions	- the e-commerce platform is online, and the search bar UI is accessible. - The search index (e.g., Elasticsearch) is running and synchronized with the main product database. - The user does not need to be logged in to execute a search.
Post conditions	Success: The user is viewing a page populated with relevant products matching their text query. Failure: The user remains on the page they searched from, viewing an error message or a "Zero Results" screen.
Normal Flow of events	- The user clicks the search bar and types in a keyword query (e.g., "mechanical keyboard"). - The system displays a dropdown of auto-complete suggestions and trending related searches as the user types. - The user submits the search query (by pressing Enter or clicking the search icon). - The system parses the text and queries the database/search engine for matching product titles, descriptions, and metadata tags. - The system sorts the matching products using a relevance algorithm. - The system displays the sorted search results to the user in a grid or list format.
Alternative Flow	4a. Misspelled Keyword: The system detects a likely typo, auto-corrects it using a fuzzy search algorithm, and displays "Showing results for [corrected word]"

	instead of zero results.
Exceptions	• Search Engine Offline: The dedicated search service (e.g., Elasticsearch) is down. The system gracefully degrades to a basic SQL database query or displays a "Search is temporarily unavailable" error.
Business rules	• Restricted Keywords: Searches for banned or illegal terms must automatically return a zero-results page and flag the search event in the Administrator's security log. • Synonym Mapping: The system must treat known synonyms equally (e.g., searching for "laptop" must also return products tagged strictly as "notebook").

2.2.3 Manage Shopping Cart

As a guest, I want to add or remove items from a temporary shopping cart so that I can review my selections before deciding to proceed with a purchase.

Use case ID	UC03
Primary Actor:	Guest user/registered user
Brief Description:	This use case describes how a user interacts with their shopping cart, including viewing added items, updating item quantities, removing items, and viewing dynamic price calculations (subtotal, estimated taxes).
Trigger	The user clicks the "Add to Cart" button from a product page or clicks the "Cart" icon in the main navigation menu to view their currently selected items.
Relationships	• Association: Guest User, Registered Customer • Include: Calculate Cart Total • Extend: Process Checkout (UC-04), Apply Promo Code • Generalization: None
Preconditions	• The e-commerce platform is operational. • The system must have a valid session ID established to track the user's temporary cart data, even if they are not logged in.
Post conditions	• Success: The cart's contents and subtotals are accurately updated in the system, and the user is ready to either continue browsing or proceed to checkout. • Failure: The cart's state remains as it was before the user attempted a modification, and an error message is displayed
Normal Flow of events	1. The user navigates to the Shopping Cart interface. 2. The system retrieves the user's active cart session from the database (or browser session for guests). 3. The system displays a list of all items currently in the cart, showing the product image, name, unit price, and current quantity. 4. The system calculates and displays the current subtotal. 5. The user modifies the cart by either: 6. Clicking "+" or "-" to change the quantity of a specific item. 7. Clicking a "Remove" or trash can icon to delete an item entirely. 8. The system immediately updates the database/session to reflect the new item state. 9. The system recalculates the subtotal dynamically and updates the display. 10. The user clicks "Proceed to Checkout" or "Continue Shopping."

Alternative Flow	• 2a. Cart is Empty: ○ The system detects no items in the active session. ○ The system displays a "Your Cart is Empty" graphic. ○ The system provides a "Start Shopping" button that redirects the user back to the main catalogue. • 5a. Exceeding Maximum Inventory: ○ The user attempts to increase the quantity of an item beyond the currently available stock (e.g., trying to buy 5 when only 3 are left). ○ The system prevents the quantity from increasing. ○ The system displays an inline warning: <i>"Only [X] items left in stock."</i> • 3a. Item Became Unavailable (Out of Stock): ○ The system loads the cart and detects that an item previously added by the user has sold out. ○ The system greys out the item and prevents checkout for that specific item. ○ The system displays an alert: <i>"An item in your cart is no longer available. Please remove it to proceed."</i>
Exceptions	• Session Timeout (Guest User): The guest user's session expires due to long inactivity. When they attempt to view the cart, the system initializes a new empty cart and notifies the user: "Your session expired and your cart has been reset."
Business rules	• Cart Expiration (Registered): Unpurchased carts belonging to Registered Customers are saved indefinitely as "Saved Carts," but their pricing is dynamically updated to reflect current database prices whenever the user logs back in (e.g., if a sale ended, the cart reflects the new higher price). • Inventory Reservation: Adding an item to the cart does <i>not</i> deduct it from the global inventory count. The inventory is only officially deducted upon successful completion of the "Process Checkout" use case.

- **2.2.4 Login**

As a Guest User, I want to log in using my registered email and password, so that I can retrieve my saved shopping cart and proceed to checkout without re-entering my details.

Use case ID	UC04
Primary Actor:	Guest user/registered user
Brief Description:	A returning user authenticates using their registered email and password to access a secure session and unlock registered features such as order history, saved cart, and account settings.
Trigger	The user clicks the 'Log In' or 'Sign In' button in the navigation, or is redirected after attempting to access a protected page or feature.
Relationships	Association: Registered Customer Include: None Extend: Register (Login may be triggered automatically upon successful registration) Generalization: None
Preconditions	The e-commerce platform and database are online. The user must already have a registered account in the system. The user must currently not be logged in.
Post conditions	Success: A secure authenticated session is created. The user is redirected to their account dashboard or the previously requested page. Failure: No session is created. The user remains on the login page and is shown an appropriate error message.
Normal Flow of events	1. The user navigates to the Login page. 2. The system displays a form requesting Email Address and Password. 3. The user enters their credentials and clicks 'Log In'. 4. The system retrieves the stored password hash associated with the provided email. 5. The system verifies the entered password against the stored hash. 6. The system creates a secure authenticated session and generates a session token. 7. The system redirects the user to their account dashboard or the previously requested page.
Alternative Flow	5a. Incorrect Password: - Hash verification fails. - Displays: 'Invalid email or password. Please try again.' (generic to prevent email enumeration). 5b. Account Does Not Exist: - No record is found for the provided email. - Displays the same generic error as 5a to prevent email enumeration.

	5c. Account Locked — Too Many Failed Attempts: - The user has exceeded the maximum allowed failed attempts (e.g., 5 consecutive failures). - Displays: 'Your account has been temporarily locked. Please reset your password or try again in 15 minutes.' 5d. Forgot Password: - The user clicks the 'Forgot Password?' link. - The system sends a time-limited password reset link to the registered email address. - The user resets their password via the link and is returned to the Login page.
Exceptions	Session token generation failure: The server fails to generate a valid session token after a successful credential check. The system logs the error internally and prompts the user to try again without revealing technical details. Concurrent login from a new device or unusual location: The system detects a new device fingerprint or geographic anomaly and sends a security alert email to the registered address before establishing the session. Database unreachable during credential lookup: The system displays 'Service temporarily unavailable. Please try again shortly.' and does not expose internal error details.
Business rules	Login Throttling: A maximum of 5 consecutive failed login attempts is permitted before the account is temporarily locked for 15 minutes. Generic Error Messages: The system must never confirm whether an email address is registered — all invalid-credential errors use the same generic message to prevent enumeration attacks. Session Expiry: Authenticated sessions expire after a configurable period of inactivity (default: 30 minutes). Secure Transmission: All login form submissions must be made over HTTPS. Credentials must never be transmitted in plain text. Remember Me: If the user opts into 'Remember Me', the system may issue a persistent token valid for a longer duration (e.g., 30 days), stored as a secure, HttpOnly cookie.

2.2.5 Register as a registered user

As a guest, I want to create a new account by providing my details so that I can save my shipping information and access order history in the future.

Use Case ID	UC05
Primary Actor:	Guest user
Brief Description:	This use case describes the process of a Guest User creating a new permanent account on the e-commerce platform by providing their personal details and setting up login credentials.
Trigger	The user clicks the "Sign Up" or "Register" button located in the site navigation, or is prompted to create an account during the checkout process
Relationships	• Association: Guest User • Include: None • Extend: Log In (The system may automatically log the user in upon successful registration) • Generalization: None
Preconditions	• The e-commerce platform and database are online. • The user must currently be recognized as a Guest User (not already logged in).
Post conditions	• Success: A new user account is created and stored securely in the database. The user is able to log in and use registered features. • Failure: No account is created, and the database remains unchanged.
Normal Flow of events	1. The user navigates to the Registration page. 2. The system displays a registration form requesting necessary details (e.g., First Name, Last Name, Email Address, and Password). 3. The user inputs their information and agrees to the Terms of Service. 4. The user clicks the "Create Account" submit button. 5. The system validates the input data formatting and checks the database to ensure the email address is not already registered. 6. The system encrypts (hashes) the password and creates a new Customer profile in the database. 7. The system generates and sends an automated welcome/verification email to the user. 8. The system displays a "Registration Successful" confirmation screen. 9. (Optional Extension) The system automatically logs the user into their new account and redirects them to the homepage or their account dashboard.

Alternative Flow	5a. Email Already Exists: • The system checks the database and finds that the provided email is already linked to an active account. • The system rejects the registration attempt. • The system displays an error message: <i>"An account with this email already exists. Please log in or reset your password."</i> 5b. Weak Password / Invalid Format: • The system validates the input and finds the password does not meet security requirements, or an email is formatted incorrectly (e.g., missing the "@" symbol). • The system highlights the specific fields with errors. • The system displays an inline warning (e.g., <i>"Password must be at least 8 characters and contain a number"</i>). 6a. Database Connection Failure: • The backend database is temporarily unreachable when the system attempts to save the new user. • The system aborts the registration and displays a standard error: <i>"We are experiencing technical difficulties. Please try again later."</i>
• Exceptions	Database write failure mid-registration: The system rolls back any partial data written and alerts the user with a generic error. No incomplete account record is left in the database.
Business rules	• Unique Identifier: The Email Address must be strictly unique across the entire database. Two accounts cannot share the same email. • Password Security: Passwords must never be stored in plain text. The system must apply a cryptographic hash function (e.g., bcrypt or Argon2) before storing the password in the Data Layer. • Data Validation: Passwords must meet a minimum complexity requirement (e.g., minimum 8 characters, at least one uppercase letter, one number, and one special character). • Terms of Service: The user must explicitly check a box confirming they agree to the platform's Terms of Service and Privacy Policy before the "Create Account" button becomes active.

2.2.6 Process Checkout

As a guest, I want to provide my shipping and payment details through a secure checkout process so that I can successfully place an order for the items in my cart.

Use Case ID	UC06
Primary Actor:	Guest user/registered user
Brief Description:	This use case describes the sequence of events from the moment a user decides to buy the items in their cart to the final confirmation of the order, including shipping calculation and payment authorization.
Trigger	The user clicks the "Proceed to Checkout" button from the Shopping Cart interface.
Relationships	• Association: Guest User, Registered Customer, Payment Gateway • Include: Verify Inventory • Extend: Apply Promo Code • Generalization: None
Preconditions	• The user must have at least one valid item in their shopping cart. • The system must have an active, secure connection to the external Payment Gateway APIs.
Post conditions	• Success: An order record is created in the database, the customer is charged, inventory is reduced, and the fulfilment module is notified to prepare for shipping. • Failure: No money is charged, no order record is finalized, and inventory counts remain unchanged.
Normal Flow of events	1. The user triggers the checkout process from the shopping cart. 2. The system verifies that the items in the cart are still in stock. 3. The system prompts the user for a shipping address (or retrieves the default address for a logged-in Registered Customer). 4. The system calculates and displays the final total, including standard shipping costs and applicable taxes. 5. The user selects a preferred payment method (e.g., Credit Card, PayPal, or FPX) and enters the required payment details. 6. The user clicks the "Place Order" button. 7. The system transmits the payment details securely to the external Payment Gateway. 8. The Payment Gateway processes the request and returns a "Success" authorization code. 9. The system generates a unique Order ID, permanently deducts the purchased items from the database inventory, and clears the user's shopping cart. 10. The system displays an Order Confirmation screen and

	emails a digital receipt to the user.
Alternative Flow	2a. Item Out of Stock: - The system verifies inventory and finds an item has sold out since it was added to the cart. - The system halts the checkout process. - The system redirects the user back to the cart with an alert: <i>"Sorry, an item in your cart is no longer available. Please remove it to continue."</i> 8a. Payment Declined / Failed: - The external Payment Gateway returns a "Declined," "Insufficient Funds," or "Timeout" error. - The system halts the order creation. - The system displays an error message: <i>"Your payment could not be processed. Please check your details or try a different payment method."</i> - The user is allowed to select a new payment method without having to re-enter their shipping details. 4a. User Applies Promo Code (Extension): - The user enters a discount code in the designated field. - The system validates the code against the database. - The system recalculates and updates the final total before the user proceeds to payment.
Exceptions	Network Interruption During Payment: If the user's browser disconnects or the system crashes exactly as the payment is processing, the system must rely on asynchronous webhooks from the Payment Gateway. If the payment actually succeeded at the bank but the user's browser crashed, the system must automatically recover the state, generate the order, and email the receipt so the user is not charged twice.
Business rules	Payment Strategy: The system must use a design pattern (like the Strategy Pattern) to route the payment to the correct gateway logic based on the user's selection, ensuring the main checkout code remains cleanly separated from external banking APIs. Tax Calculation: Sales tax must be dynamically calculated based on the destination postal/zip code entered in the shipping address, not the user's current billing address. Order Immutability: Once an order is placed and the Payment Gateway returns a success code, the order details (price paid, items bought) are locked. Any further changes require triggering a separate "Refund/Return" use case.

2.2.7 Track Order

As a guest, I want to check the status of my shipment using a unique order ID so that I can know exactly when my delivery will arrive.

Use Case ID	UC07
Primary Actor:	Guest user/registered user
Brief Description:	This use case describes how a user retrieves the current shipping status, location history, and estimated delivery date of a previously placed order.
Trigger	The user clicks the "Track Order" button from their digital receipt email, or clicks "Track Package" next to an item in their Account Dashboard.
Relationships	• Association: Guest User, Registered Customer, Courier API • Include: None • Extend: None • Generalization: None
Preconditions	• The user must have a valid Order ID that exists in the system database. • The system must have active integration with the relevant third-party logistics (3PL) or Courier APIs.
Post conditions	• Success: The user successfully views their package's current status and location timeline. • Failure: The user is denied access due to invalid credentials, or views an error regarding API connectivity.
Normal Flow of events	1. The user navigates to the "Track Order" interface (either via an email link or their account dashboard). 2. The system prompts the user for their Order ID and billing email address (Note: If the user is a logged-in Registered Customer, the system skips this step and retrieves it automatically). 3. The user inputs the requested details and clicks "Search". 4. The system validates the input against the database and retrieves the internal order record and associated Courier Tracking Number. 5. The system sends a secure request containing the tracking number to the external Courier API (e.g., DHL, NinjaVan, J&T Express). 6. The Courier API responds with the latest package location, status (e.g., "In Transit", "Out for Delivery"), and a timestamped history log. 7. The system formats this data and displays a visual timeline and estimated delivery date to the user.
Alternative Flow	4a. Invalid Order ID / Email Mismatch: • The system checks the database and cannot find an order

	matching the provided ID and email combination. • The system rejects the request to protect user privacy. • The system displays an error message: <i>"Order not found. Please check your Order ID and email address and try again."</i> 6a. Order Not Yet Shipped (Pre-Transit): • The system retrieves the internal order but finds no tracking number has been generated yet because warehouse staff are still packing it. • The system bypasses the Courier API. • The system displays an internal status: <i>"Processing: Your order is being prepared for shipment."</i> 6b. Courier API Timeout / Unreachable: • The system pings the external Courier API, but the courier's servers are down or timing out. • The system displays the last known status saved in the local database. • The system displays an alert: <i>"Live tracking is temporarily unavailable from the courier. Showing last known status."</i>
• Exceptions	• Order ID belongs to a different user: The system detects a session mismatch, denies access, and logs the attempt as a security event. • Courier API returns malformed data: The system catches the parsing error, falls back to the last cached status, and alerts the operations team internally. • Database unreachable during validation: Displays 'Service temporarily unavailable.' without exposing internal error details.
Business rules	• Data Privacy: Tracking details (especially the destination address) must never be accessible using only an Order ID. A secondary point of authentication (like the buyer's email or being logged into the purchasing account) is strictly required to prevent malicious actors from scraping delivery addresses. • Status Mapping: The e-commerce system must map the courier's specific status codes (which vary by company) to a unified, user-friendly format (e.g., mapping FedEx's "On FedEx vehicle for delivery" and DHL's "With delivery courier" to a standard "Out for Delivery" status). • Delivery Confirmation: When the Courier API returns a "Delivered" status, the e-commerce system must automatically trigger a background process to update the internal database to "Completed" and email a delivery confirmation/review request to the user.

Actor 2 Administrator

2.3.1 Manage Inventory

As an admin, I want to update stock levels and product descriptions to ensure the storefront is accurate.

Use Case ID	UCO8
Primary Actor:	Administrator
Brief Description:	This use case describes the process by which an authorized Administrator adds new products to the catalogue, updates existing product details (such as price, description, or stock quantity), or removes discontinued items from the platform.
Trigger	The Administrator clicks on the "Inventory Management" tab from their secure Admin Dashboard.
Relationships	• Association: Administrator • Include: None • Extend: None • Generalization: None
Preconditions	• The user must be successfully authenticated and have an active session with "Administrator" privileges. • The backend database and inventory management services must be online.
Post conditions	• Success: The database is successfully updated with the new product information, and the changes are immediately pushed to the customer-facing catalogue. • Failure: The database is unchanged, the Administrator is notified of the specific error preventing the update.
Normal Flow of events	1. The Administrator navigates to the Inventory Management dashboard. 2. The system retrieves and displays a paginated list of all current products in the database. 3. The Administrator searches for a specific product using its unique SKU (Stock Keeping Unit) or title. 4. The system displays the specific product's current details. 5. The Administrator clicks "Edit" and modifies the desired fields (e.g., increasing the stock count from 0 to 50, or updating the price). 6. The Administrator clicks "Save Changes". 7. The system validates the new input data (e.g., ensuring the price is a positive number). 8. The system updates the product record in the Data Layer. 9. The system immediately refreshes the search index so the updated stock/price reflects on the live storefront. 10. The system displays a "Product Updated Successfully" confirmation message.

Alternative Flow	3a. Product SKU Not Found: [occurs at step 3] - The SKU searched does not exist in the database. - Displays: 'SKU not found. Would you like to add a new product?' 7a. Invalid Data Entry: [occurs at step 7] - The Administrator enters an invalid value (e.g., negative stock quantity). - Highlights the incorrect field in red. - Displays: 'Stock quantity cannot be negative. Please enter a valid number.'
• Exceptions	Concurrent Editing Conflict: If two Administrators attempt to edit the exact same product at the exact same time, the system must employ "Optimistic Locking." The first Admin to click save succeeds. When the second Admin clicks save, the system rejects it and alerts them: <i>"This product was just modified by another user. Please refresh to see the latest changes before editing."</i>
Business rules	• Soft Deletion: An Administrator cannot permanently delete a product from the database if it is linked to any past customer orders (this would break the order history receipts). Instead, clicking "Delete" must simply flag the product's status as "Archived" or "Inactive," hiding it from the public catalog while preserving the data. • SKU Uniqueness: Every product and variant (e.g., a Red Shirt vs. a Blue Shirt) must have a strictly unique SKU. The system must reject any attempt to create a new product using an existing SKU. • Audit Logging: Every time an Administrator adds, edits, or archives a product, the system must automatically log the Admin's User ID, the timestamp, and the exact changes made into a secure Audit Log for security and tracking purposes.

2.3.2 Manage Promotions

As an admin, I want to create and expire discount codes to drive seasonal sales

Use Case ID	UC09
Primary Actor:	Administrator
Brief Description:	This use case describes how an Administrator creates, configures, and activates promotional discount codes or site-wide sales events within the e-commerce platform..
Trigger	The Administrator clicks on the "Promotions & Marketing" tab from their secure Admin Dashboard and selects "Create New Promotion."
Relationships	• Association: Administrator • Include: None • Extend: None • Generalization: None
Preconditions	• The user must be authenticated with "Administrator" or "Marketing" privileges. • The system's pricing and discount calculation engine must be online.
Post conditions	• Success: The new promotion is saved in the database and the system's checkout module is now prepared to accept and validate the code from customers. • Failure: The database remains unchanged, and the Administrator is notified of the specific validation error.
Normal Flow of events	1. The Administrator navigates to the Promotions dashboard. 2. The system retrieves and displays a list of all active, scheduled, and expired promotions. 3. The Administrator clicks the "Create New Promo Code" button. 4. The system displays a blank promotion configuration form. 5. The Administrator inputs the promotion parameters: • Promo Code String (e.g., "SUMMER20") • Discount Type (Percentage vs. Flat Amount) • Discount Value (e.g., 20% off) • Start Date and Expiry Date • Usage Limits (e.g., "First 100 customers only" or "Minimum spend \$50") 6. The Administrator clicks "Save and Activate." 7. The system validates the input (e.g., ensuring the expiry date is in the future). 8. The system saves the new promotional rule to the Data Layer. 9. The system displays a "Promotion Successfully Created" confirmation message.

Alternative Flow	7a. Duplicate Promo Code: - The Administrator enters a promo code string (e.g., "WELCOME10") that already exists in the database for a past or current campaign. - The system detects the duplicate during validation. - The system highlights the code field in red. - The system displays an error message: <i>"This promo code already exists. Please enter a unique code."</i> and prevents saving. 7b. Invalid Date Configuration: - The Administrator sets an Expiry Date that occurs <i>before</i> the Start Date. - The system catches the logical error during validation. - The system displays an alert: <i>"Expiry date must be after the start date."</i>
Exceptions	Database Lock/Timeout: If the database is under heavy load (e.g., during a major site-wide sale) and the Administrator attempts to update a live promotion, the system must queue the update request and notify the Admin once the change has successfully propagated across the servers.
Business rules	Discount Constraints: A percentage-based discount cannot exceed 100%. A flat-amount discount cannot reduce a cart's subtotal below \$0.00. Code Formatting: Promo codes are automatically converted to uppercase by the system to prevent case-sensitivity errors when customers type them in at checkout. Non-Stacking Rule: By default, the system enforces a rule that only one promotional code can be applied per shopping cart. If a customer attempts to add a second code, the system must prompt them to choose which single discount they prefer to use. Category Exclusions: The Administrator must be able to flag certain products (e.g., "New Arrivals" or "Gift Cards") to be strictly excluded from all promotional discounts.

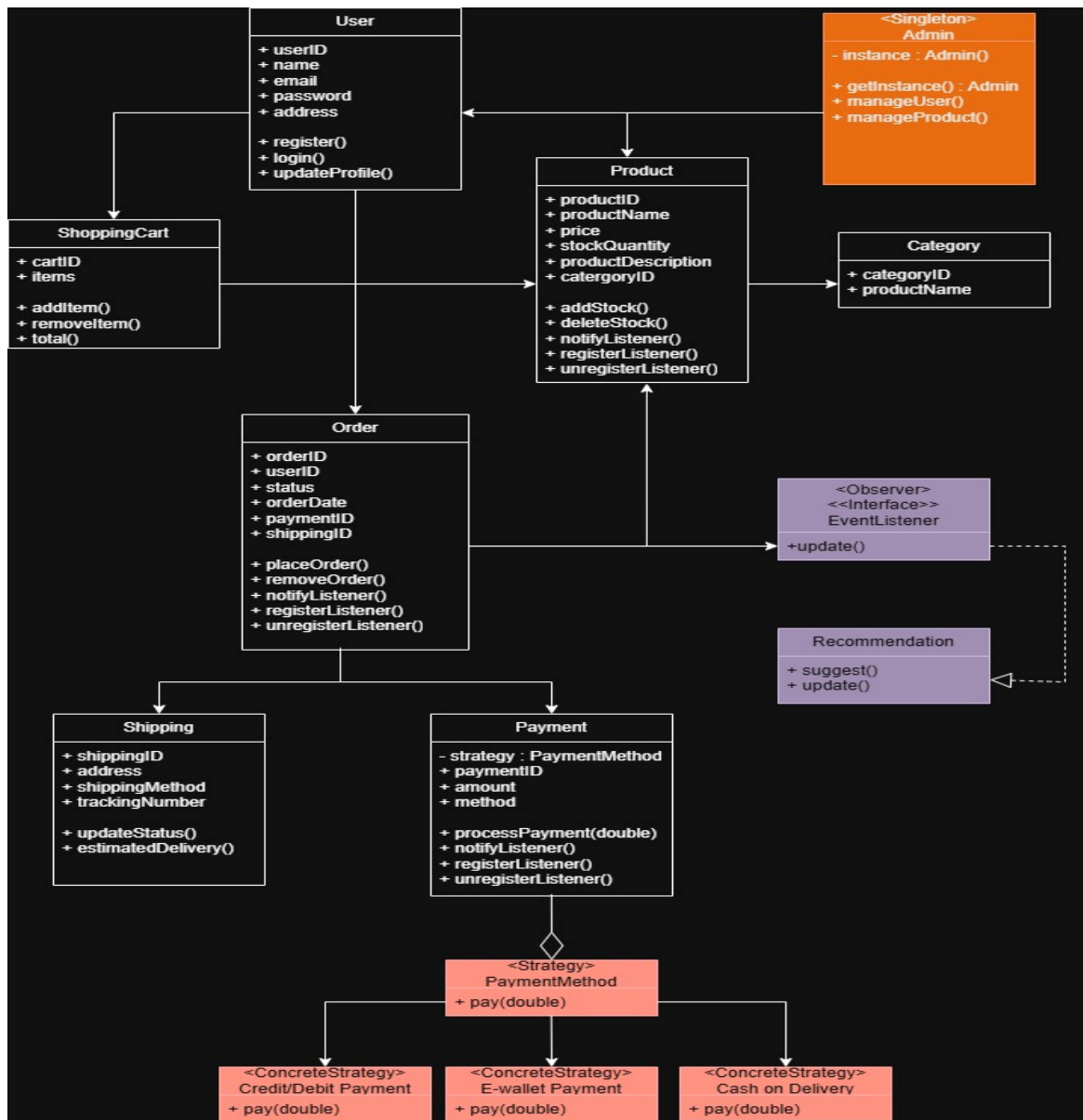
2.3.3 Generate Reports

As an admin, I want to view sales performance and user metrics to make informed business decisions.

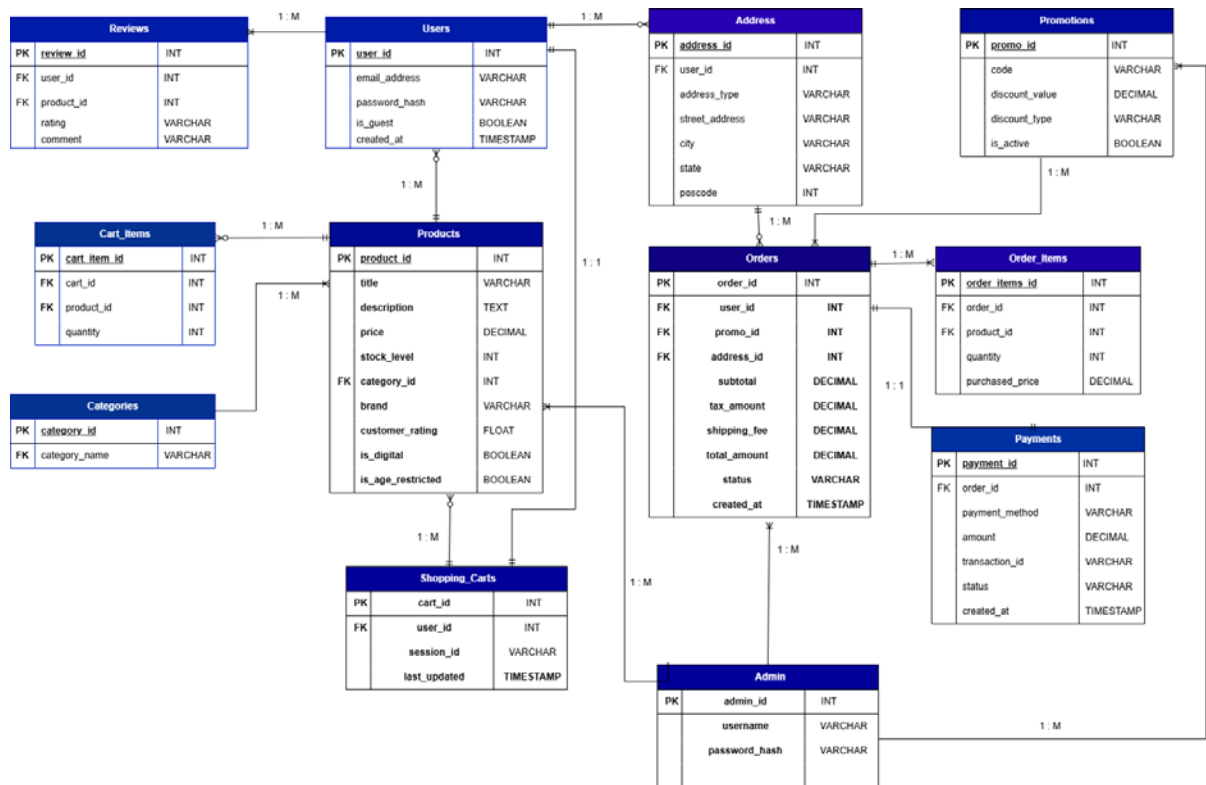
Primary Actor:	Administrator
Brief Description:	This use case describes how an Administrator generates, views, and exports business intelligence reports (e.g., sales data, inventory levels) from the system.
Trigger	The Administrator clicks the "Reports" or "Analytics" tab in the Admin Dashboard.
Relationships	• Association: Administrator • Include: None • Extend: None • Generalization: None
Preconditions	• The Administrator must be logged in with sufficient privileges. • The database must contain historical transaction and inventory data.
Post conditions	• Success: The requested report is generated and displayed or downloaded. • Failure: The system fails to generate the report due to a timeout or lack of data.
Normal Flow of events	1. The Administrator navigates to the Reports dashboard. 2. The system presents various report templates (Sales, Inventory, Traffic). 3. The Administrator selects a report type and sets date range filters. 4. The Administrator clicks "Generate Report". 5. The system queries the database and aggregates the data. 6. The system displays the report in a chart or table format. 7. The Administrator clicks "Export to CSV/PDF". 8. The system compiles the file and triggers the download.
• Alternative Flow	5a. No Data in Date Range: The system finds no records for the selected dates and displays <i>"No data available for the selected period."</i> Exceptions: • Query Timeout: The requested dataset is too large, causing the database query to time out. The system aborts the process and suggests narrowing the date range. Business rules:

	• Data Accuracy: Reports must reflect real-time data up to the moment of generation. • Privacy: Exported reports must mask sensitive customer data (like passwords or full credit card numbers).
• Exceptions	Query Timeout: The dataset is too large, causing the query to exceed its time limit. - Displays: 'Report generation timed out. Please narrow your date range and try again.' - The failed query is logged internally for performance monitoring. Export Engine Failure: The CSV/PDF compiler fails to produce the file. - Displays: 'Export failed. Please try again or contact support.' - The on-screen report remains visible.
Business rules	• Data Accuracy: Reports must reflect real-time data up to the exact moment of generation. • Privacy: Exported reports must mask sensitive customer data (passwords, full credit card numbers). • Access Control: Only users with Administrator or Analyst privileges may access the Reports module. • Audit Trail: Each report generation is logged with Admin User ID, timestamp, report type, and date range.

2.4 Class Diagram

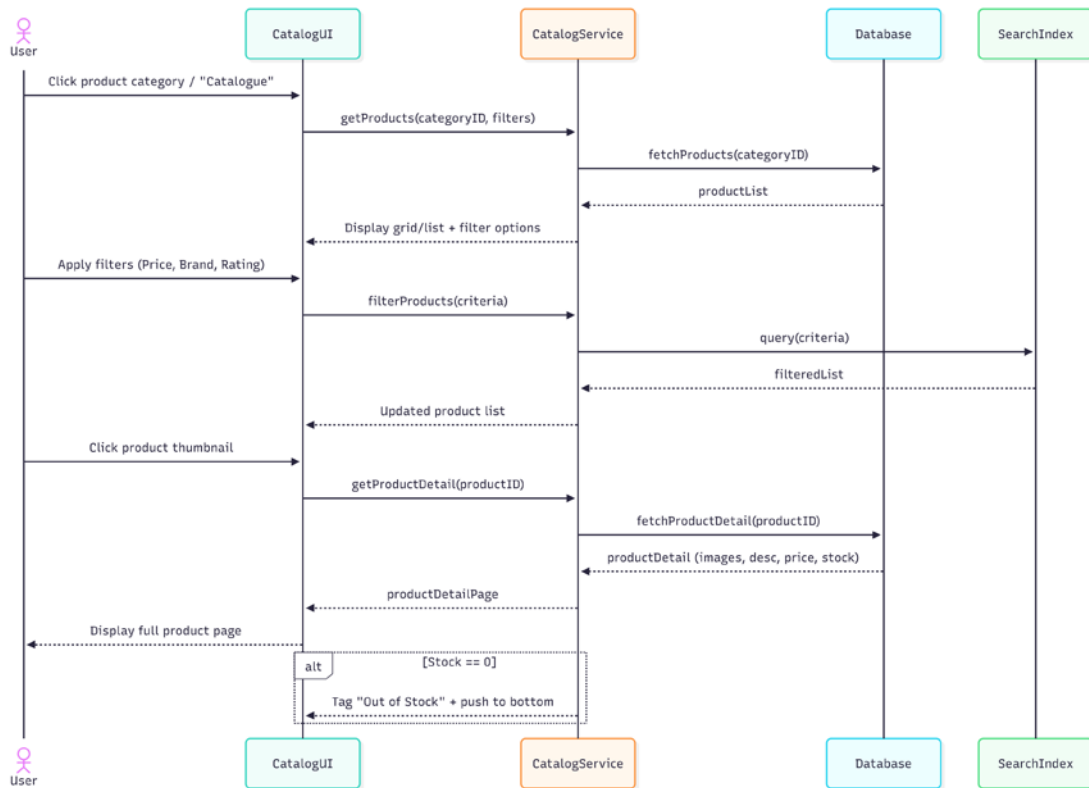


2.5 ERD

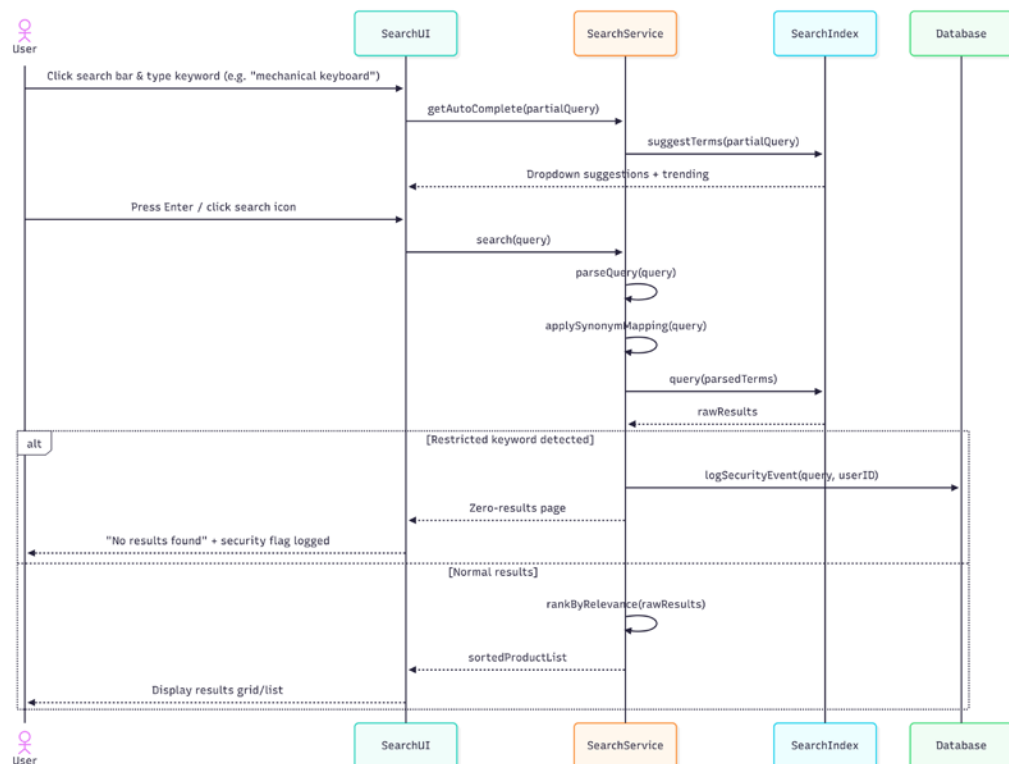


2.5 Sequence diagram

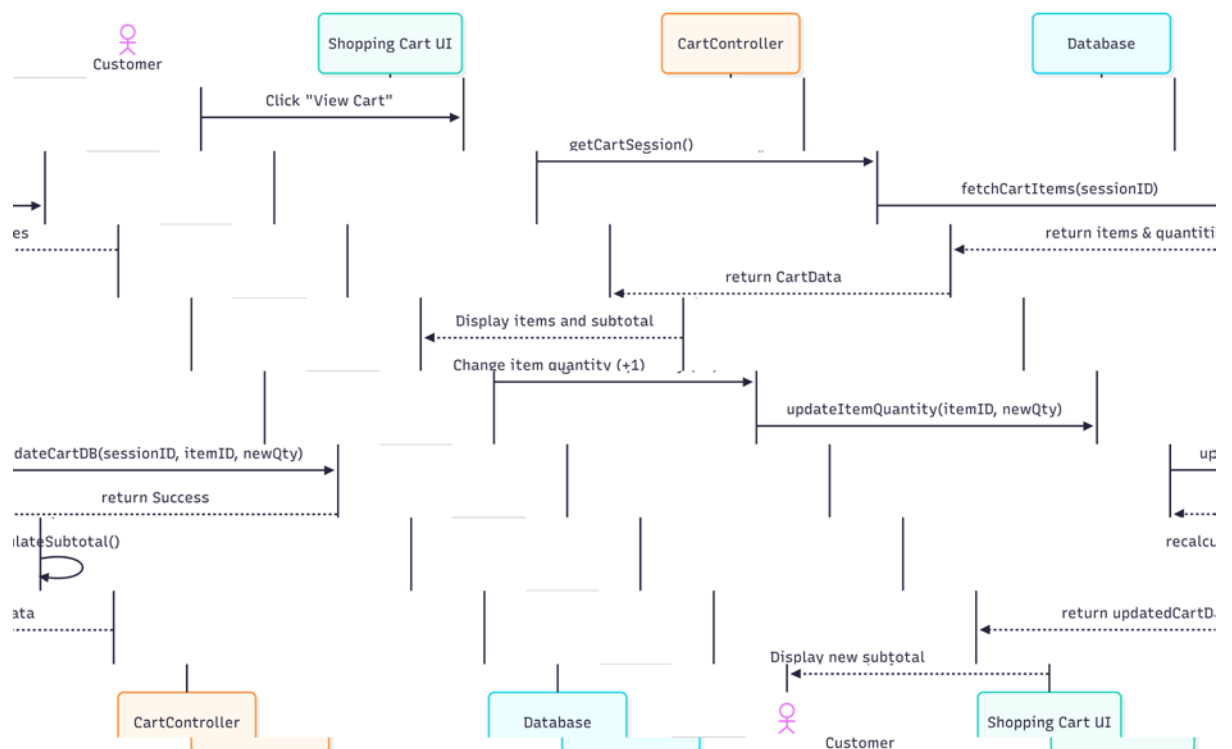
2.5.1 Browse catalogue



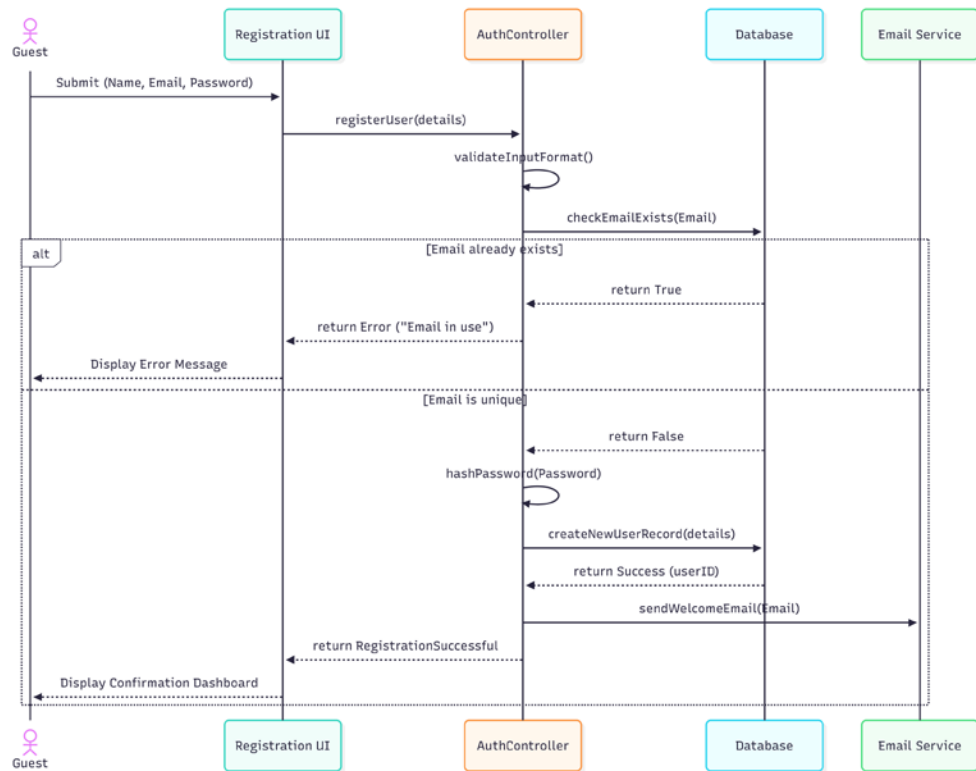
2.5.2 Search products



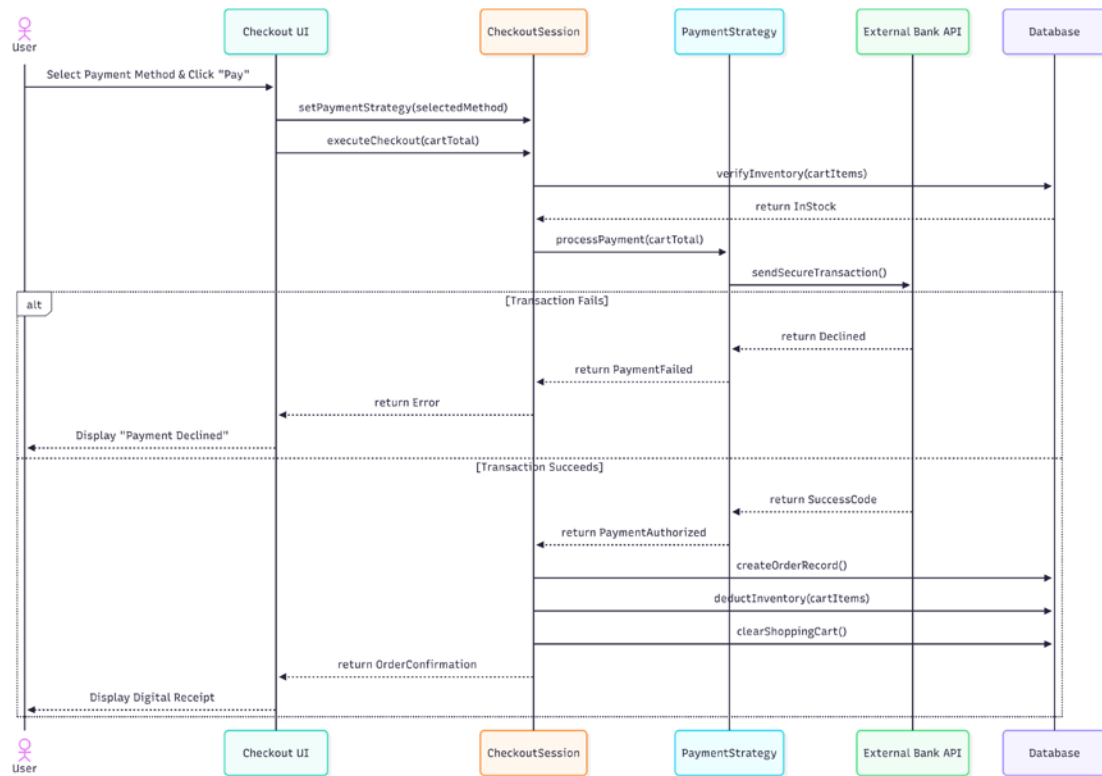
2.5.3 Manage Shopping Cart



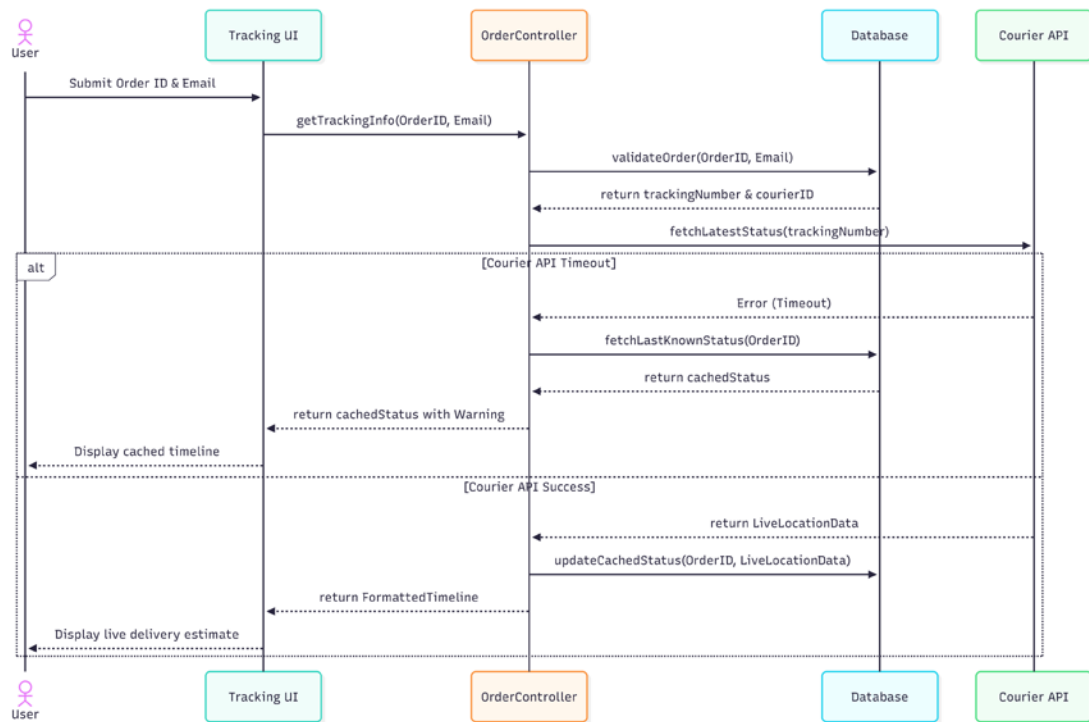
2.5.4 Register User



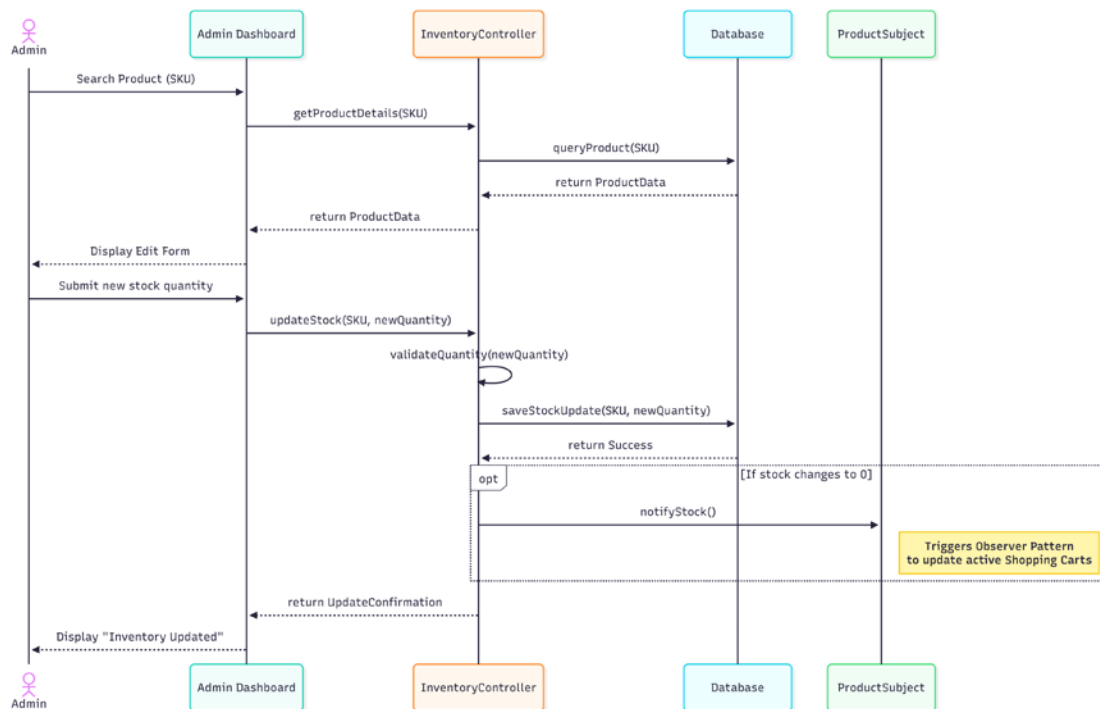
2.5.5 Process Checkout (Applying Strategy Pattern)



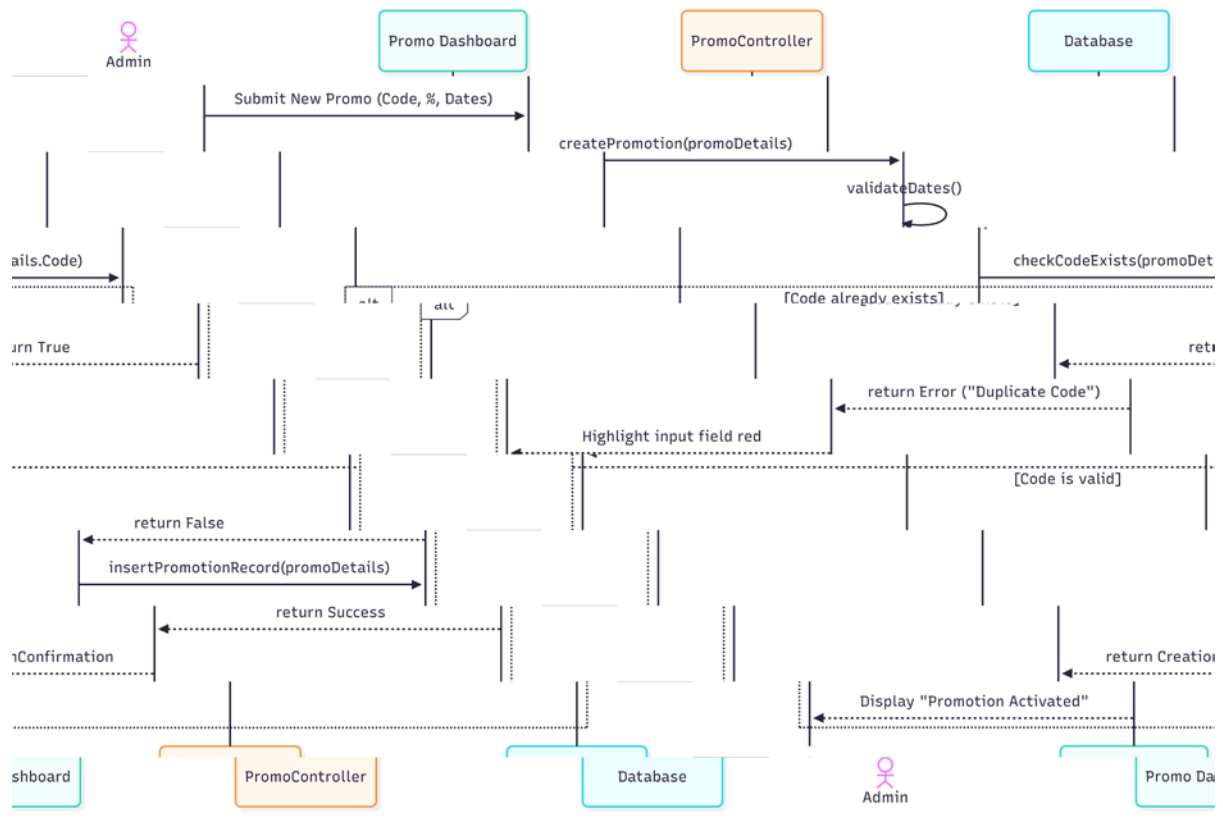
2.5.6 Track Order



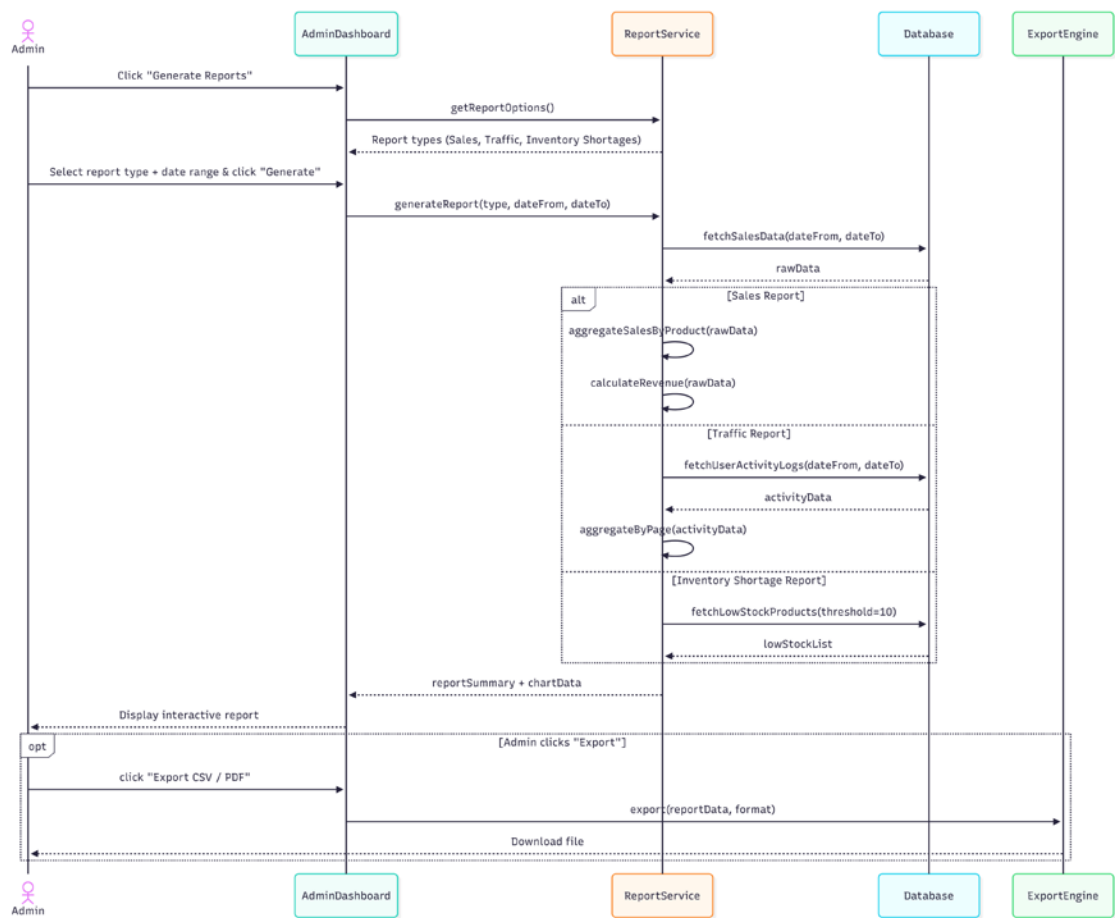
2.5.7 Manage Inventory(Admin)



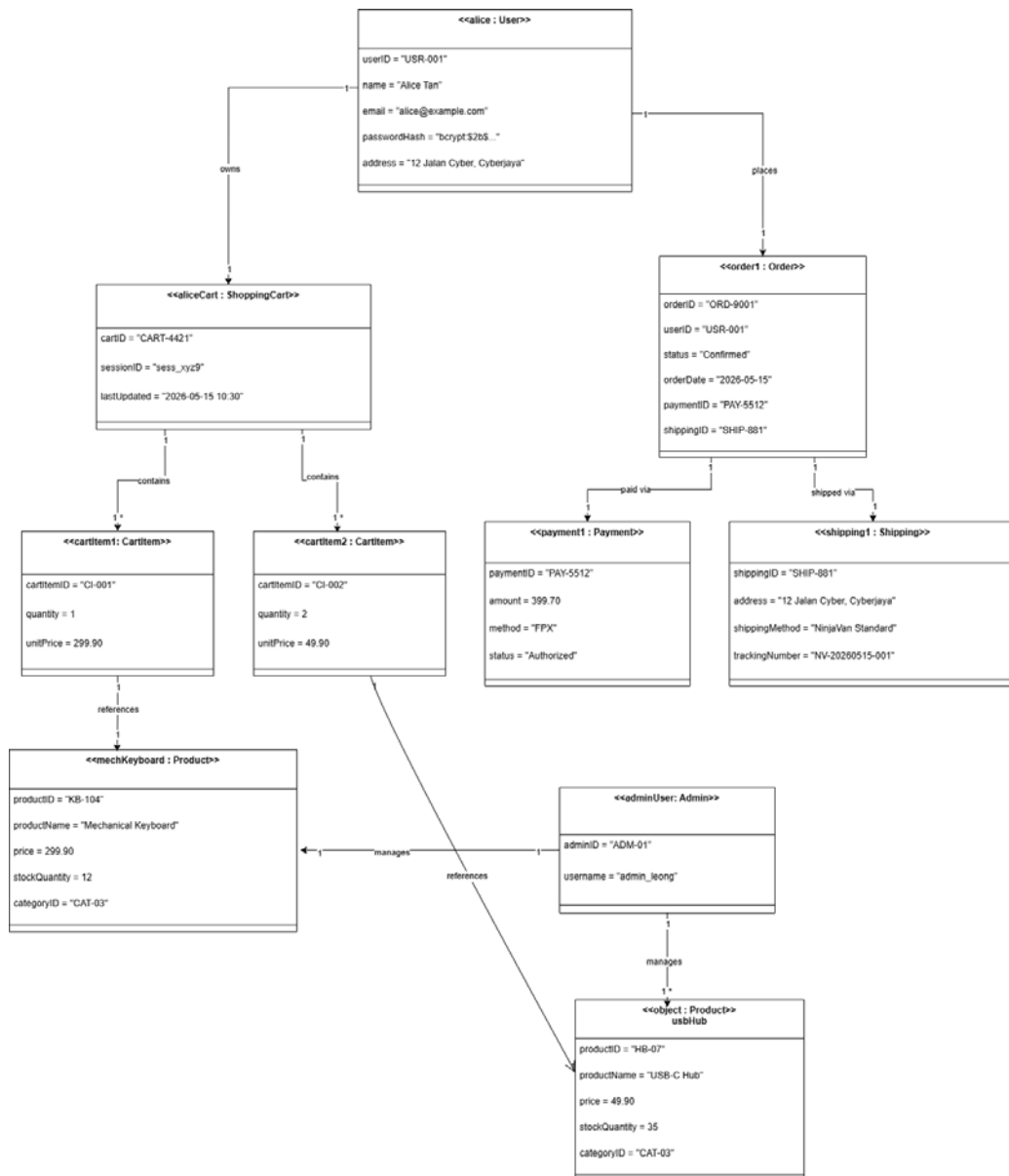
2.5.8 Manage Promotions(Admin)



2.5.9 Generate Reports (Admin)



2.6 Object diagram



3. Requirements

Software Design Concepts and Design Principles

3.1 The SOLID Principles

3.1.1 S-Single responsibility principle

This is the most famous set of object-oriented guidelines, formulated by Robert C. Martin (Uncle Bob):

- **S - Single Responsibility Principle (SRP):**
Applied to: User Registration module (MVC + Three-Tier Architecture)
Rule: Each class has exactly one reason to change.

The registration flow is split into four dedicated classes: User (data model), UserValidator (input rules), UserRepository (database persistence), and UserMailer (email notifications). The UserRegistrationService orchestrates them. Changing the email provider never touches the validator; changing password rules never touches the database layer.

- User — holds user data only
- UserValidator — validates email format, password complexity
- UserRepository — persists and queries user records
- UserMailer — sends transactional emails
- UserRegistrationService — orchestrates the workflow

```
<?php

// =====
// S – SINGLE RESPONSIBILITY PRINCIPLE (SRP)
// Each class has ONE reason to change.
// User data, validation, DB persistence, and email are
// separated into dedicated classes.
// =====

class User
{
    public function __construct(
        public readonly string $name,
        public readonly string $email,
        public readonly string $passwordHash,
        public readonly string $address = ''
```


) {}
}
// Responsibility 1: Input validation only
class UserValidator
{
public function validate(array \$data): array
{
\$errors = [];
if (empty(\$data['name']))
\$errors[] = 'Name is required.';
if (!filter_var(\$data['email'] ?? '', FILTER_VALIDATE_EMAIL))
\$errors[] = 'A valid email address is required.';
if (strlen(\$data['password'] ?? '') < 8)
\$errors[] = 'Password must be at least 8 characters.';
if (!preg_match('/[A-Z]/', \$data['password'] ?? ''))
\$errors[] = 'Password must contain at least one uppercase letter.';
if (!preg_match('/[0-9]/', \$data['password'] ?? ''))
\$errors[] = 'Password must contain at least one number.';
return \$errors;
}
}
// Responsibility 2: Database persistence only
class UserRepository
{
private array \$storage = [];
public function emailExists(string \$email): bool
{
foreach (\$this->storage as \$user)
if (\$user->email === \$email) return true;
return false;
}
public function save(User \$user): bool

{
\$this->storage[] = \$user;
echo "[DB] User '{\$user->name}' saved to database.\n";
return true;
}
}
// Responsibility 3: Email notifications only
class UserMailer
{
public function sendWelcomeEmail(User \$user): void
{
echo "[Mailer] Welcome email sent to {\$user->email}.\n";
}
}
// Responsibility 4: Registration workflow orchestration only
class UserRegistrationService
{
public function __construct(
private UserValidator \$validator,
private UserRepository \$repository,
private UserMailer \$mailer
) {}
public function register(array \$data): string
{
\$errors = \$this->validator->validate(\$data);
if (!empty(\$errors))
return 'Registration failed: ' . implode(' ', \$errors);
if (\$this->repository->emailExists(\$data['email']))
return 'Registration failed: Email already in use.';
\$user = new User(
name: \$data['name'],

email:	\$data['email'],
passwordHash:	password_hash(\$data['password'], PASSWORD_BCRYPT)
	);
	\$this->repository->save(\$user);
	\$this->mailer->sendWelcomeEmail(\$user);
	return 'Registration successful!';
	}
	}
	// --- Usage ---
	\$service = new UserRegistrationService(
	new UserValidator(),
	new UserRepository(),
	new UserMailer()
	);
	echo \$service->register([
	'name' => 'Alice',
	'email' => ' alice@example.com ',
	'password' => 'Secure123',
	]) . "\n";
	?>

3.1.2 O - Open/Closed Principle (OCP):

Applied to: Checkout & Payment Processing subsystem (Strategy Pattern)

Rule: Open for extension, closed for modification.

The PaymentStrategy interface defines the pay() contract. Each payment method (Credit Card, E-Wallet, FPX, Cash on Delivery) is a separate class implementing that interface. CheckoutSession is permanently closed — adding Apple Pay next year requires only a new class, zero changes to existing code.

- PaymentStrategy — interface defining pay() and getMethodName()
- CreditCardPayment, EWalletPayment, FPXBankingPayment — concrete strategies
- CashOnDeliveryPayment — extension added without modifying CheckoutSession
- CheckoutSession — context class, NEVER modified when new payment methods are added

```
<?php

// =====
// O – OPEN/CLOSED PRINCIPLE (OCP)
// Open for extension, closed for modification.
// Add new payment methods without touching CheckoutSession.
// =====

interface PaymentStrategy
{
    public function pay(float $amount): bool;
    public function getMethodName(): string;
}

class CreditCardPayment implements PaymentStrategy
{
    public function __construct(private string $cardNumber) {}
    public function pay(float $amount): bool
    {
        echo "[Credit Card] Charging RM{$amount} to card ending "
            . substr($this->cardNumber, -4) . ".\n";
        return true;
    }
    public function getMethodName(): string { return 'Credit/Debit Card'; }
```

}
class EWalletPayment implements PaymentStrategy
{
public function __construct(private string \$walletId) {}
public function pay(float \$amount): bool
{
echo "[E-Wallet] Transferring RM{\$amount} from wallet {\$this->walletId}.\n";
return true;
}
public function getMethodName(): string { return 'E-Wallet'; }
}
class FPXBankingPayment implements PaymentStrategy
{
public function __construct(private string \$bankCode) {}
public function pay(float \$amount): bool
{
echo "[FPX] Routing RM{\$amount} via FPX to bank {\$this->bankCode}.\n";
return true;
}
public function getMethodName(): string { return 'FPX Online Banking'; }
}
// EXTENSION: new method added WITHOUT modifying CheckoutSession
class CashOnDeliveryPayment implements PaymentStrategy
{
public function pay(float \$amount): bool
{
echo "[COD] Customer will pay RM{\$amount} upon delivery.\n";
return true;
}
public function getMethodName(): string { return 'Cash on Delivery'; }
}
// Context class – CLOSED for modification

class CheckoutSession
{
private ?PaymentStrategy \$strategy = null;
public function __construct(private float \$cartTotal) {}
public function setPaymentStrategy(PaymentStrategy \$strategy): void
{
\$this->strategy = \$strategy;
}
public function executeCheckout(): bool
{
if (\$this->strategy === null) {
echo "[Checkout] Error: No payment method selected.\n";
return false;
}
echo "[Checkout] Processing via {\$this->strategy->getMethodName()}...\n";
\$result = \$this->strategy->pay(\$this->cartTotal);
if (\$result) echo "[Checkout] Order confirmed. Receipt emailed.\n";
return \$result;
}
}
// --- Usage ---
\$checkout = new CheckoutSession(250.00);
\$checkout->setPaymentStrategy(new FPXBankingPayment('MBB0293'));
\$checkout->executeCheckout();
?>

3.1.3 L - Liskov Substitution Principle (LSP):

Applied to: Observer Pattern — Inventory & real-time stock notifications

Rule: Subtypes must be substitutable for their base type without altering correctness.

Product (Subject) holds a list of StockObserver references. At runtime those slots may hold a ShoppingCart, a Wishlist, or a RecommendationEngine. The Product calls observer->update() on every one — it never checks which concrete type it is dealing with. Each concrete observer fulfills the same contract and can be swapped freely.

- StockObserver — interface with update(productId, newStock)
- ShoppingCart — greys out item and blocks checkout when stock hits 0
- Wishlist — alerts user when a saved item goes out of stock or comes back
- RecommendationEngine — removes out-of-stock items from suggestions
- Product — Subject; notifies all attached observers on stock change

```
<?php

// =====
// L - LISKOV SUBSTITUTION PRINCIPLE (LSP)
// Subtypes must be substitutable for their base type.
// Product (Subject) notifies any StockObserver - ShoppingCart,
// Wishlist, or RecommendationEngine - interchangeably.
// =====

interface StockObserver
{
    public function update(string $productId, int $newStock): void;
}

// Concrete Observer A - Shopping Cart
class ShoppingCart implements StockObserver
{
    private array $items = [];

    public function __construct(private string $cartId) {}

    public function addItem(string $productId, int $qty): void
    {
        $this->items[$productId] = $qty;
    }
}
```

}
public function update(string \$productId, int \$newStock): void
{
if (isset(\$this->items[\$productId]) && \$newStock === 0)
echo "[Cart {\$this->cartId}] ALERT: '{\$productId}' is OUT OF STOCK. "
. "Remove it to proceed to checkout.\n";
}
}
// Concrete Observer B – Wishlist
class Wishlist implements StockObserver
{
public function __construct(private string \$userId) {}
public function update(string \$productId, int \$newStock): void
{
if (\$newStock === 0)
echo "[Wishlist:{\$this->userId}] '{\$productId}' is now out of stock.\n";
elseif (\$newStock > 0)
echo "[Wishlist:{\$this->userId}] '{\$productId}' is back in stock!
({\$newStock} units)\n";
}
}
// Concrete Observer C – Recommendation Engine
class RecommendationEngine implements StockObserver
{
public function update(string \$productId, int \$newStock): void
{
if (\$newStock === 0)
echo "[Recommendations] Removing '{\$productId}' from suggested items.\n";
}
}
// Subject – works with ANY StockObserver via the interface
class Product


```

{
    private array $observers = [];

    public function __construct(
        public readonly string $productId,
        public readonly string $name,
        private int $stockQuantity
    ) {}

    public function attach(StockObserver $observer): void
    {
        $this->observers[] = $observer;
    }

    public function detach(StockObserver $observer): void
    {
        $this->observers = array_filter(
            $this->observers, fn($o) => $o !== $observer
        );
    }

    private function notifyStock(): void
    {
        foreach ($this->observers as $observer)
            $observer->update($this->productId, $this->stockQuantity);
    }

    public function setStockQuantity(int $qty): void
    {
        $this->stockQuantity = max(0, $qty);
        echo "\n[Inventory] '{$this->name}' stock set to {$this->stockQuantity}.\n";
        $this->notifyStock();
    }
}

// --- Usage ---

```

```
$keyboard = new Product('KB-104', 'Mechanical Keyboard', 5);
```

```
$aliceCart = new ShoppingCart('cart_alice');
```

```
$aliceCart->addItem('KB-104', 1);
```

```
$keyboard->attach($aliceCart);
```

```
$keyboard->attach(new Wishlist('alice'));
```

```
$keyboard->attach(new RecommendationEngine());
```

```
$keyboard->setStockQuantity(0); // All observers notified
```

```
?>
```

3.1.4 I - Interface Segregation Principle (ISP):

Applied to: External API integrations (Payment Gateway, Courier API, Report Generator)

Rule: No class should be forced to implement methods it does not use.

Instead of a single monolithic ExternalServices interface, the system defines three thin, purpose-specific interfaces. CourierServices depends only on PaymentGateway. NinjaVanCourier depends only on CourierService. Admin reporting depends only on ReportGenerator. Concrete classes (Stripe, NinjaVan) implement only the interface they serve.

- PaymentGateway — sendTransaction(), refundTransaction()
- CourierService — getTrackingStatus(), estimateDelivery()
- ReportGenerator — generateSalesReport(), generateInventoryReport()
- StripeGateway — implements PaymentGateway only
- NinjaVanCourier — implements CourierService only
- CheckoutProcessor — depends on PaymentGateway only
- OrderTracker — depends on CourierService only

```
<?php

// =====
// I – INTERFACE SEGREGATION PRINCIPLE (ISP)
// No class is forced to implement methods it does not use.
// PaymentGateway, CourierService, and ReportGenerator are
// separate interfaces – each module depends only on its own.
// =====

// Segregated interface 1: Payment only
interface PaymentGateway
{
    public function sendTransaction(string $orderId, float $amount, string $method):
bool;
    public function refundTransaction(string $transactionId, float $amount): bool;
}

// Segregated interface 2: Shipping / tracking only
interface CourierService
{
    public function getTrackingStatus(string $trackingNumber): array;
    public function estimateDelivery(string $trackingNumber): string;
}
```

// Segregated interface 3: Reporting only (Admin)
interface ReportGenerator
{
public function generateSalesReport(string \$from, string \$to): array;
public function generateInventoryReport(): array;
}
// Stripe implements ONLY PaymentGateway
class StripeGateway implements PaymentGateway
{
public function sendTransaction(string \$orderId, float \$amount, string \$method): bool
{
echo "[Stripe] Charging RM{\$amount} for Order#{ \$orderId} via { \$method}.\n";
return true;
}
public function refundTransaction(string \$transactionId, float \$amount): bool
{
echo "[Stripe] Refunding RM{\$amount} for transaction { \$transactionId}.\n";
return true;
}
}
// NinjaVan implements ONLY CourierService
class NinjaVanCourier implements CourierService
{
public function getTrackingStatus(string \$trackingNumber): array
{
return [
'tracking_number' => \$trackingNumber,
'status' => 'Out for Delivery',
'location' => 'Cyberjaya Hub',
'updated_at' => date('Y-m-d H:i:s'),
];
}
public function estimateDelivery(string \$trackingNumber): string
{

return date('Y-m-d', strtotime('+1 day'));
}
}
// CheckoutProcessor depends ONLY on PaymentGateway (ISP)
class CheckoutProcessor
{
public function __construct(private PaymentGateway \$gateway) {}
public function processOrder(string \$orderId, float \$total, string \$method): void
{
\$success = \$this->gateway->sendTransaction(\$orderId, \$total, \$method);
echo \$success
? "[Checkout] Payment successful for Order#{\$orderId}.\n"
: "[Checkout] Payment failed for Order#{\$orderId}.\n";
}
}
// OrderTracker depends ONLY on CourierService (ISP)
class OrderTracker
{
public function __construct(private CourierService \$courier) {}
public function track(string \$orderId, string \$trackingNumber): void
{
\$status = \$this->courier->getTrackingStatus(\$trackingNumber);
\$eta = \$this->courier->estimateDelivery(\$trackingNumber);
echo "[Tracking] Order#{\$orderId} – Status: {\$status['status']} "
. "at {\$status['location']}. ETA: {\$eta}.\n";
}
}
// --- Usage ---
\$processor = new CheckoutProcessor(new StripeGateway());
\$processor->processOrder('ORD-9001', 299.90, 'Credit Card');

```
$tracker = new OrderTracker(new NinjaVanCourier());
```

```
$tracker->track('ORD-9001', 'NV-20250513-001');
```

```
?>
```

3.1.5 D - Dependency Inversion Principle (DIP):

Applied to: Order Processing pipeline — OrderService high-level module

Rule: High-level modules must not depend on low-level modules. Both must depend on abstractions.

OrderService is the high-level orchestrator for placing and tracking orders. It never calls new StripeGateway() or new MySQLOrderRepository() internally. Instead, all four dependencies (OrderRepository, PaymentGateway, CourierService, NotificationService) are injected through the constructor as interfaces. This means the entire payment vendor, database engine, or notification channel can be swapped at runtime with zero changes to OrderService.

- OrderRepository — abstraction for order persistence (implemented by MySQLOrderRepository)
- NotificationService — abstraction for notifications (implemented by EmailNotificationService)
- PaymentGateway — abstraction for payments (same as OCP/ISP sections)
- CourierService — abstraction for tracking (same as ISP section)
- OrderService — high-level module; depends on interfaces only via constructor injection

```
<?php

// =====
// D – DEPENDENCY INVERSION PRINCIPLE (DIP)
// High-level modules depend on abstractions, not concretions.
// OrderService depends only on interfaces – concrete classes
// (Stripe, NinjaVan, MySQL) are injected at runtime.
// =====

// (Interfaces PaymentGateway & CourierService defined in ISP section)

// Abstraction: Order persistence
interface OrderRepository
{
    public function create(array $orderData): string;
    public function updateStatus(string $orderId, string $status): void;
    public function findById(string $orderId): ?array;
```

```

}

// Abstraction: Notification channel
interface NotificationService
{
    public function send(string $recipient, string $subject, string $message): void;
}

// Low-level concrete: MySQL
class MySQLOrderRepository implements OrderRepository
{
    private array $db = [];

    public function create(array $orderData): string
    {
        $orderId = 'ORD-' . strtoupper(substr(md5(uniqid()), 0, 6));
        $this->db[$orderId] = array_merge($orderData, [
            'id' => $orderId, 'status' => 'Pending',
            'created_at' => date('Y-m-d H:i:s'),
        ]);
        echo "[MySQL] Order {$orderId} inserted into orders table.\n";
        return $orderId;
    }

    public function updateStatus(string $orderId, string $status): void
    {
        if (isset($this->db[$orderId])) {
            $this->db[$orderId]['status'] = $status;
            echo "[MySQL] Order {$orderId} status → '{$status}'.\n";
        }
    }

    public function findById(string $orderId): ?array
    {
        return $this->db[$orderId] ?? null;
    }
}

```


}
// Low-level concrete: Email
class EmailNotificationService implements NotificationService
{
public function send(string \$recipient, string \$subject, string \$message): void
{
echo "[Email] To: {\$recipient} Subject: {\$subject}\n";
echo " Message: {\$message}\n";
}
}
// HIGH-LEVEL MODULE – depends on abstractions only (DIP)
class OrderService
{
public function __construct(
private OrderRepository \$orderRepo, // abstraction
private PaymentGateway \$payment, // abstraction
private CourierService \$courier, // abstraction
private NotificationService \$notifier // abstraction
) {}
public function placeOrder(
string \$customerEmail,
array \$cartItems,
float \$total,
string \$paymentMethod
): string {
// 1. Persist order
\$orderId = \$this->orderRepo->create([
'customer_email' => \$customerEmail,
'items' => \$cartItems,
'total' => \$total,
'payment_method' => \$paymentMethod,
]);

// 2. Process payment via injected gateway
\$paid = \$this->payment->sendTransaction(\$orderId, \$total, \$paymentMethod);
if (!\$paid) {
\$this->orderRepo->updateStatus(\$orderId, 'Payment Failed');
return "Order {\$orderId}: payment failed.";
}
// 3. Confirm order
\$this->orderRepo->updateStatus(\$orderId, 'Confirmed');
// 4. Notify customer via injected service
\$this->notifier->send(
\$customerEmail,
"Order Confirmed - {\$orderId}",
"Thank you! Your order {\$orderId} of RM{\$total} has been confirmed."
);
return "Order {\$orderId} placed successfully!";
}
public function trackOrder(string \$orderId, string \$trackingNumber): void
{
\$order = \$this->orderRepo->findById(\$orderId);
if (\$order === null) { echo "[OrderService] Order not found.\n"; return; }
\$status = \$this->courier->getTrackingStatus(\$trackingNumber);
echo "[OrderService] {\$status['status']} at {\$status['location']}. \n";
}
}
// --- Usage (all dependencies INJECTED – no new Stripe() inside OrderService) ---
\$orderService = new OrderService(
orderRepo: new MySQLOrderRepository(),
payment: new StripeGateway(),
courier: new NinjaVanCourier(),
notifier: new EmailNotificationService()
);

\$result = \$orderService->placeOrder(
customerEmail: ' alice@example.com ',
cartItems: [['sku' => 'KB-104', 'qty' => 1, 'price' => 299.90]],
total: 299.90,
paymentMethod: 'Credit Card'
);
echo "[Result] {\$result}\n";
?>

4. Proposed Design Patterns

4.1 Design Pattern 1: The Strategy Pattern

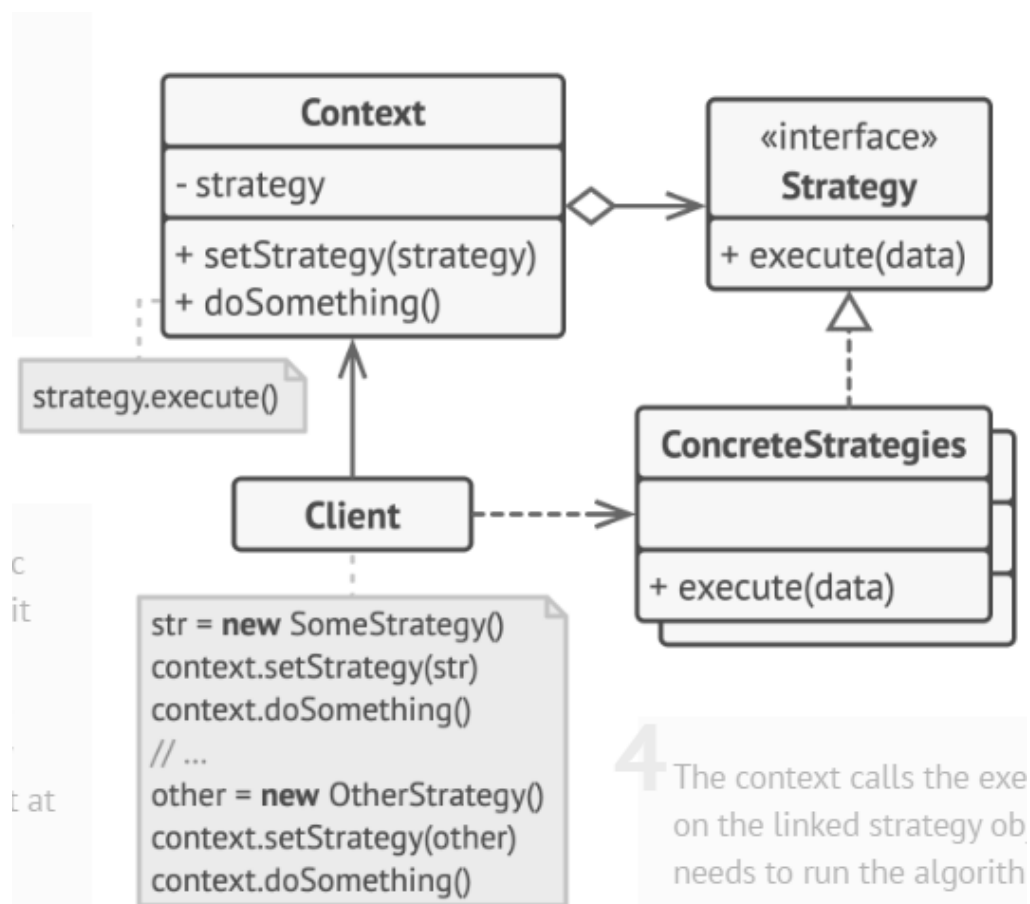
- **Module:** Checkout & Payment Processing subsystem.
- **Components:** Checkout Session class, External Payment Gateway interfaces.

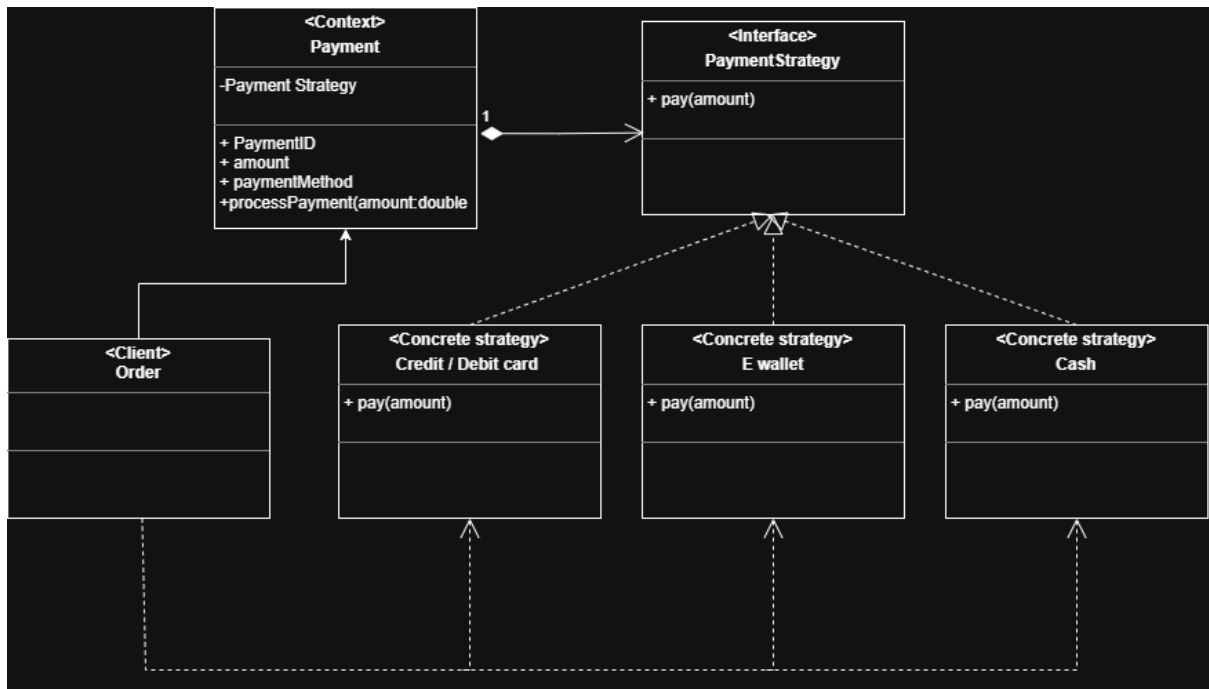
4.1.1 Description of workflow or data flow

The Strategy Pattern defines a family of algorithms (payment methods), encapsulates each one, and makes them interchangeable at runtime.

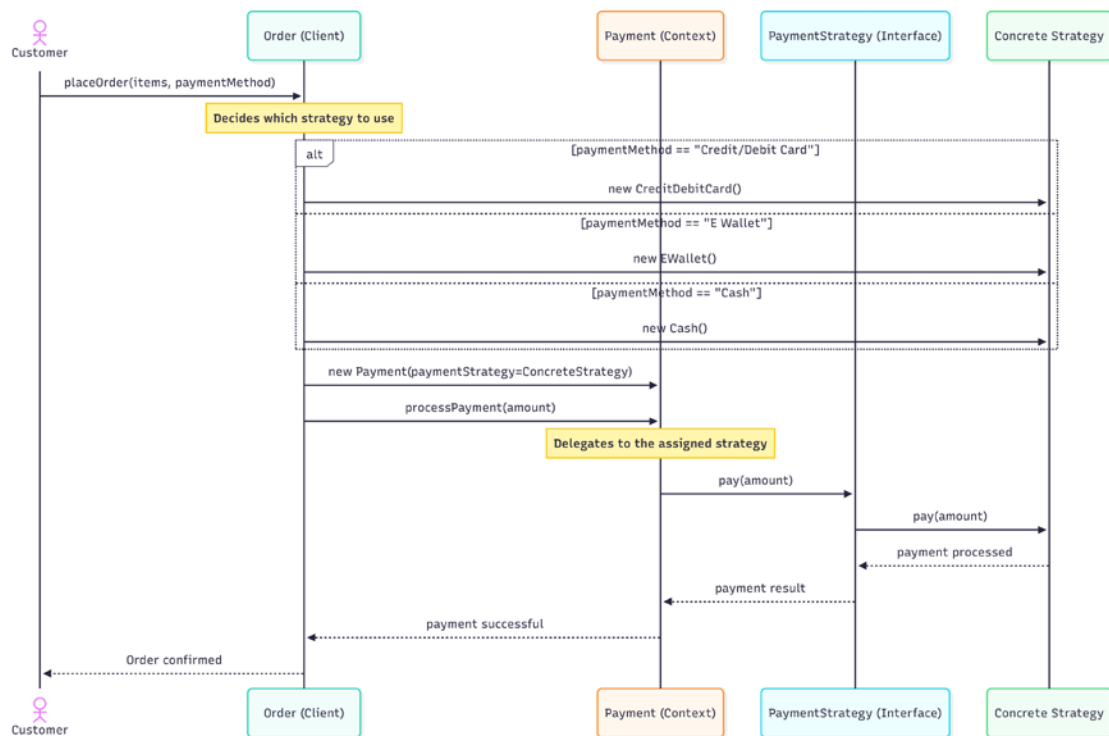
1. **Data Flow:** The user reaches the final checkout screen, and the system calculates the final totalAmount.
2. **Workflow:** The user selects a specific payment method from the UI (e.g., Credit Card or E-Wallet).
3. **Execution:** Instead of running through massive if/else statements, the CheckoutSession dynamically instantiates the specific payment class selected by the user. It then calls a generic processPayment() command. The system routes the data to the correct external API without the core checkout module needing to know the complex details of how that specific bank processes the transaction.

4.1.2 Sample Class Diagram





4.1.3 Sample Other UML Notations (Sequence Diagram)



4.1.4 Sample Potential Code

```
<?php

// 1. The Strategy Interface
interface PaymentStrategy {
    public function processPayment(float $amount): bool;
}

// 2. Concrete Strategy A: Credit Card
class CreditCardPayment implements PaymentStrategy {
    private string $cardNumber;

    public function __construct(string $cardNumber) {
        $this->cardNumber = $cardNumber;
    }

    public function processPayment(float $amount): bool {
        echo "Routing RM{$amount} to Credit Card API for card: {$this->cardNumber}\n";
        // Logic to connect to Visa/Mastercard would go here
        return true;
    }
}

// 3. Concrete Strategy B: FPX Online Banking
class FPXBankingPayment implements PaymentStrategy {
    private string $bankID;

    public function __construct(string $bankID) {
        $this->bankID = $bankID;
    }

    public function processPayment(float $amount): bool {
        echo "Routing RM{$amount} to FPX Gateway for Bank ID: {$this->bankID}\n";
        // Logic to connect to Maybank2U/CIMB Clicks would go here
    }
}
```

return true;
}
}
// 4. The Context Class: Checkout Session
class CheckoutSession {
private ?PaymentStrategy \$currentStrategy = null;
private float \$cartTotal;
public function __construct(float \$total) {
\$this->cartTotal = \$total;
}
// Dynamically change the algorithm at runtime
public function setPaymentStrategy(PaymentStrategy \$strategy): void {
\$this->currentStrategy = \$strategy;
}
public function executeCheckout(): void {
if (\$this->currentStrategy === null) {
echo "Error: No payment method selected.\n";
return;
}
echo "Initiating checkout...\n";
\$this->currentStrategy->processPayment(\$this->cartTotal);
}
}
// --- Execution Example ---
\$session = new CheckoutSession(150.50);
// User selects FPX Banking
\$session->setPaymentStrategy(new FPXBankingPayment('MBB0293'));
\$session->executeCheckout();

?>

4.1.5. Benefit

- **Open/Closed Principle (OCP):** The system is open for extension but closed for modification. If the business decides to add a new payment method (e.g., Apple Pay) next year, developers simply create a new Apple Pay class. They do not have to rewrite or touch the existing CheckoutSession code, drastically reducing the risk of introducing bugs into the checkout pipeline.
- **High Cohesion and Loose Coupling:** The checkout module focuses strictly on managing the cart state, while the payment logic is entirely decoupled and isolated into its own dedicated classes.

4.1.6. Limitations

- **Increased Class Count:** Because every single payment method requires its own dedicated class file, the overall number of classes in the system's architecture increases, which can slightly elevate the initial structural complexity.
- **Client Awareness:** The client (in this case, the User Interface or controller) must be aware of the different payment strategies in order to instantiate the correct one before passing it to the checkout session.

4.2 Design Pattern 2: The Observer Pattern

4.2.1. Function or Software Component Affected

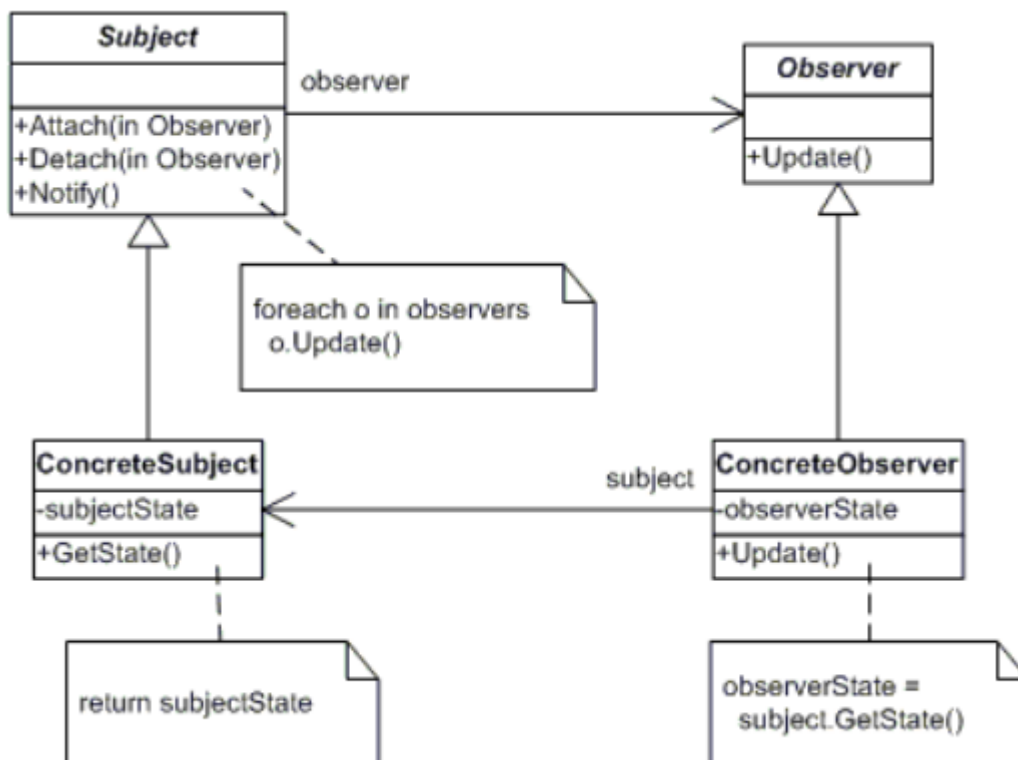
- **Module:** Catalog Management and Shopping Cart subsystems.
- **Components:** Product class (Subject), Observer interface, and ShoppingCart class (Concrete Observer).

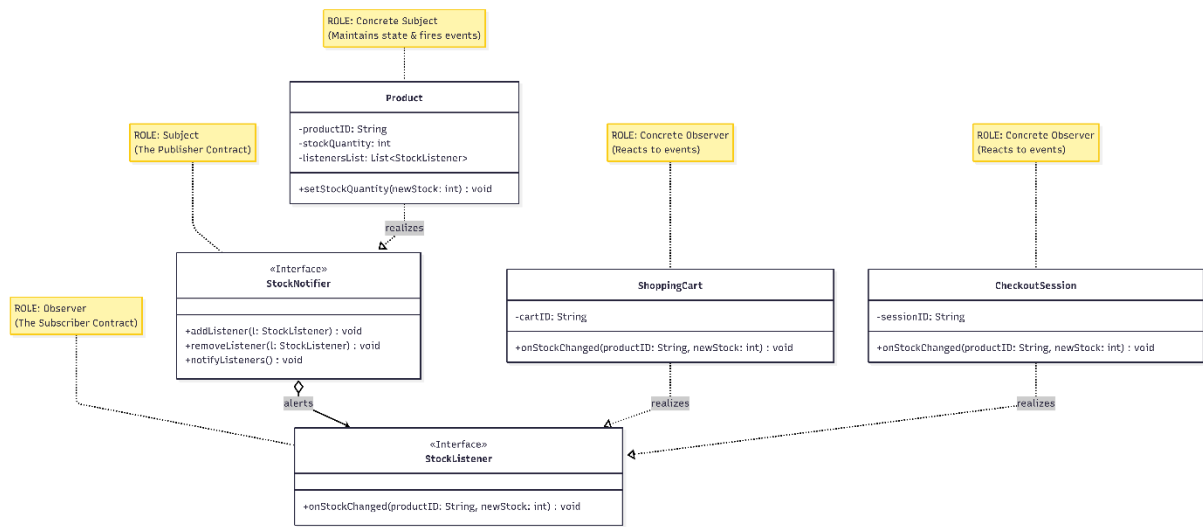
4.2.2. Description of workflow or data flow

The Observer Pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

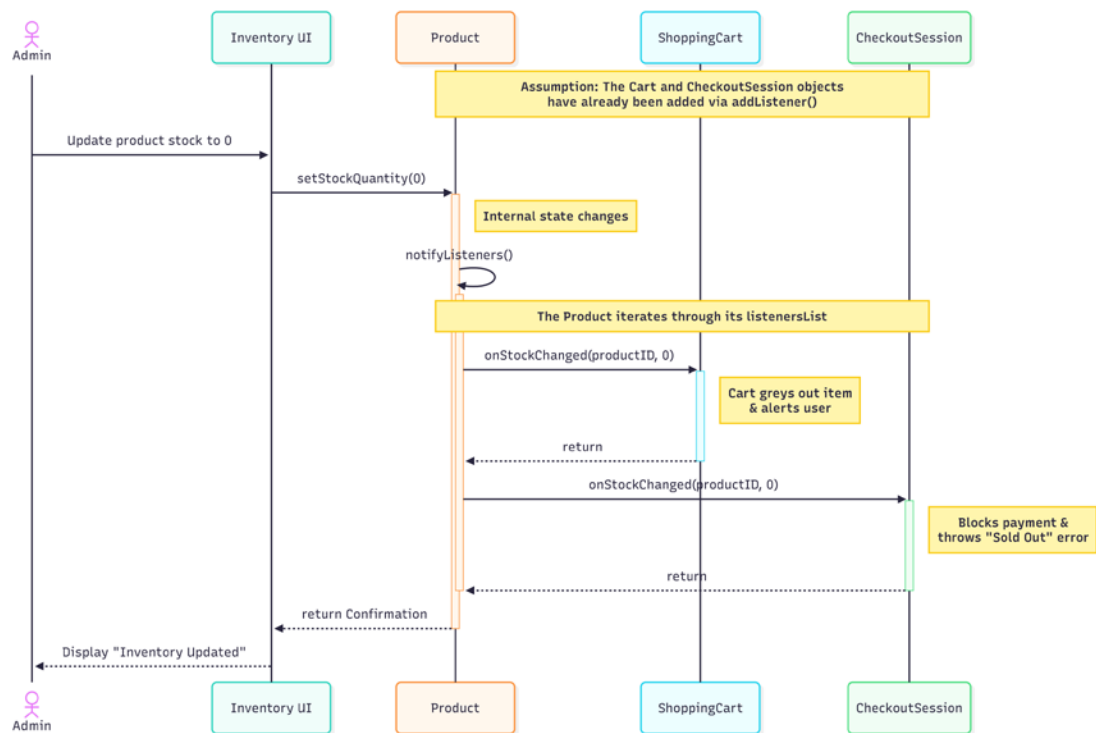
1. Data Flow: An Admin updates the stockQuantity of a specific Product in the database.
2. Workflow: As users browse the site, their active ShoppingCart objects register themselves by calling attach(Observer) on the Product objects they contain.
3. Execution: When the Product's stock drops to zero, the Product object triggers its notifyStock() method. This iterates through a list of all attached Observers and calls their update() method. The ShoppingCart receives this signal and automatically updates its UI to alert the user that the item is no longer available, preventing checkout errors.

4.2.3. Sample Class Diagram





4.2.4 Sample Other UML Notations (Sequence Diagram)



4.2.5. Sample Potential Code

```

// 1. The Observer Interface
interface Observer {
    public function update(string $productID, int $newStock): void;
}

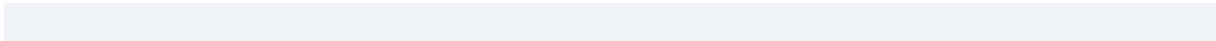
// 2. The Subject Class: Product
class Product {
    private string $productID;
    private int $stockQuantity;
    private array $observersList = [];

    public function __construct(string $productID, int $stock) {
        $this->productID = $productID;
        $this->stockQuantity = $stock;
    }
}

```

public function attach(Observer \$o): void {
\$this->observersList[] = \$o;
}
public function detach(Observer \$o): void {
\$this->observersList = array_filter(
\$this->observersList,
fn(\$obs) => \$obs !== \$o
);
}
public function notifyStock(): void {
foreach (\$this->observersList as \$observer) {
\$observer->update(\$this->productID, \$this->stockQuantity);
}
}
// Changing stock triggers the notification
public function setStockQuantity(int \$newStock): void {
\$this->stockQuantity = \$newStock;
echo "\n[System] Product {\$this->productID} stock updated to: {\$newStock}\n";
\$this->notifyStock();
}
}
// 3. Concrete Observer: Shopping Cart
class ShoppingCart implements Observer {
private string \$cartID;
public function __construct(string \$cartID) {
\$this->cartID = \$cartID;
}

public function update(string \$productID, int \$newStock): void {
if (\$newStock === 0) {
echo "[Cart {\$this->cartID}] Alert: Item {\$productID} in your cart is now ";
echo "OUT OF STOCK. Please remove it to checkout.\n";
} else {
echo "[Cart {\$this->cartID}] Note: Item {\$productID} stock has changed. ";
echo "Available: {\$newStock}\n";
}
}
}



4.2.6. Benefit

- **Real-Time Data Consistency:** Ensures that thousands of concurrent users see accurate stock limitations in their active shopping carts without needing to refresh the page.
- **Reduced Server Load:** The system does not require active polling (where the cart constantly asks the database "Is this in stock?"). The cart only reacts when explicitly notified by the product, conserving massive amounts of processing power.
- **Loose Coupling:** The Product class does not need to know the complex internal details of a ShoppingCart or a Recommendation engine. It only knows that it must send a signal to any object implementing the Observer interface.

4.2.7. Limitations

- **The Lapsed Listener Problem (Memory Leaks):** If a user abandons their session and the ShoppingCart object is destroyed, but the system forgets to call detach(Observer), the Product will continue trying to notify a non-existent cart, leading to memory leaks or program crashes.
- **Cascading Updates:** If an Admin updates the stock of 1,000 items in bulk, and each item has 5,000 observers attached, it can trigger a sudden flood of 5,000,000 update() calls, potentially causing a temporary spike in CPU usage.

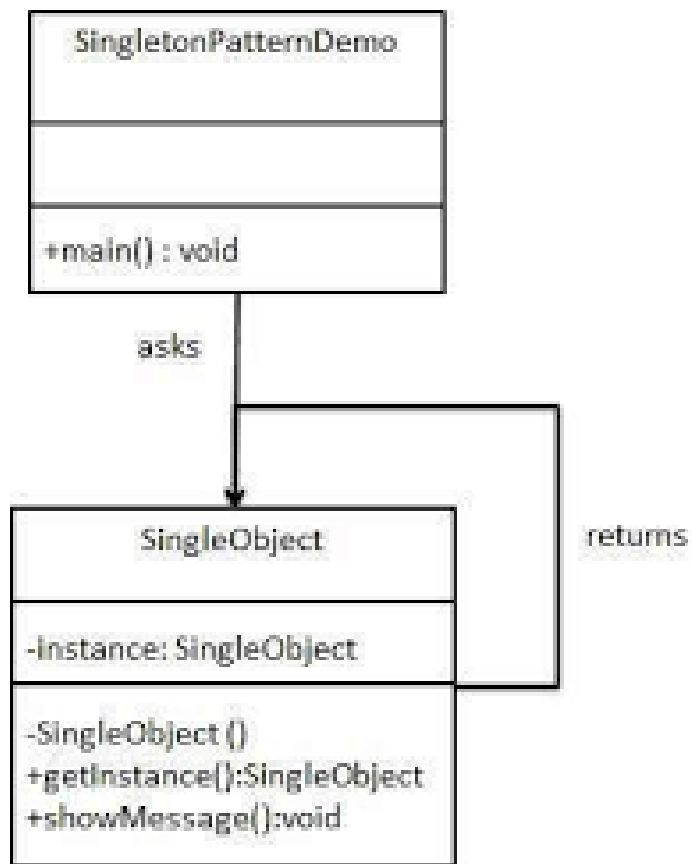
4.3 Design Pattern 3: The Singleton Pattern

4.3.1. Description of workflow or data flow

The Singleton Pattern is a creational design pattern that restricts the instantiation of a class to one "single" instance.

1. **Data Flow:** The system initializes, but the Admin object is not created immediately to save memory (Lazy Initialization).
2. **Workflow:** When an administrative action is required (e.g., updating stock, creating a promo code), the UI or backend module requests access to the Admin manager by calling the static `getInstance()` method.
3. **Execution:** The class checks if its private internal instance property is null. If it is null, it creates the one and only Admin object. If it already exists, it simply returns the reference to the existing object. This guarantees that all admin operations across the entire e-commerce platform are routed through the exact same centralized instance.

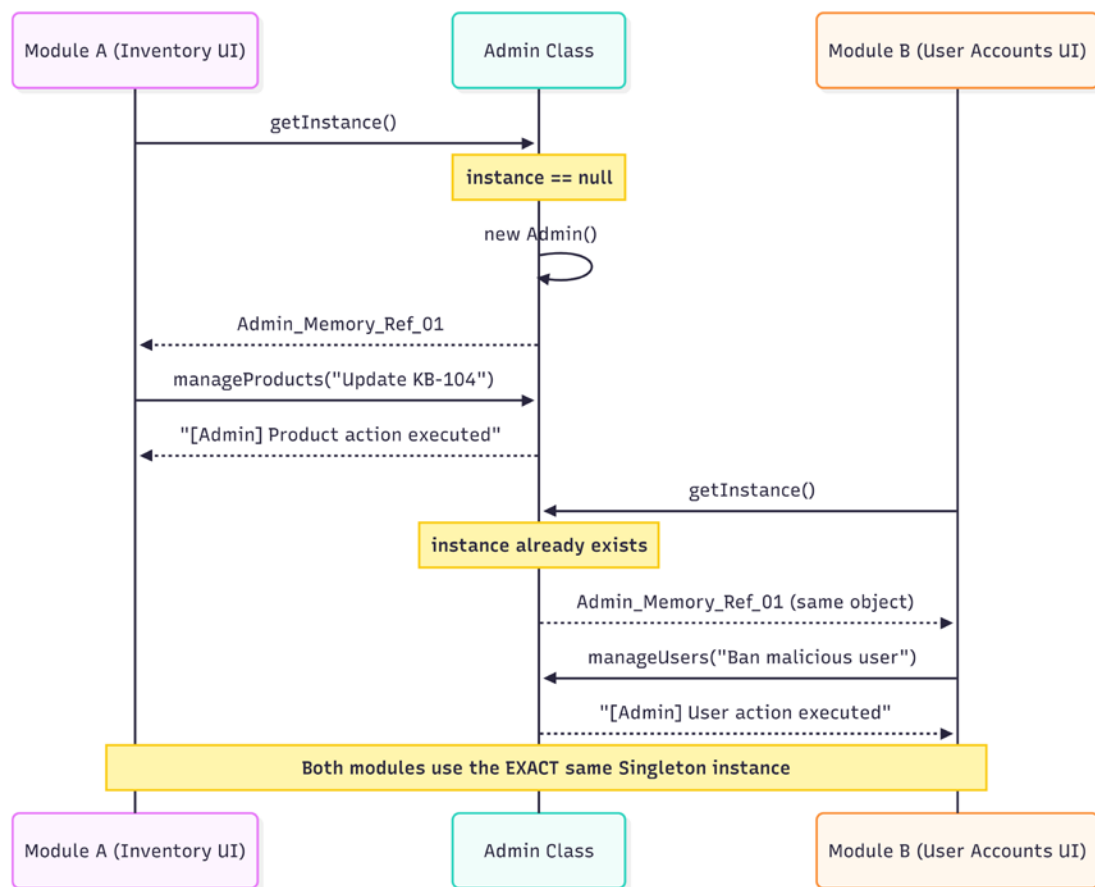
4.3.2 Sample Class Diagram



The constructor is private (-) to prevent normal instantiation. Access is strictly through the static getInstance() method.



4.3.3. Sample Other UML Notations (Sequence Diagram)



A **Sequence Diagram** is best to show how multiple system modules interact with the Singleton over time.

1. **Module A (Inventory UI)** sends `getInstance()` to the **Admin Class**.
2. **Admin Class** checks its state. instance is null.
3. **Admin Class** instantiates new `Admin()` and returns **Admin_Memory_Ref_01**.
4. **Module B (User Accounts UI)** sends `getInstance()` to the **Admin Class**.
5. **Admin Class** checks its state. instance already exists.
6. **Admin Class** skips creation and simply returns the exact same **Admin_Memory_Ref_01**.

4.3.4 Sample Potential Code

Here is how the classic Singleton architecture translates into C++ code:

```
<?php

class Admin {

    // 1. Private static instance property
    private static ?Admin $instance = null;

    // 2. Private constructor prevents direct instantiation
    private function __construct() {
        echo "[System] Admin instance created in memory.\n";
    }

    // 3. Prevent cloning to enforce Singleton rule
    private function __clone() {}

    // 4. Prevent unserialization to enforce Singleton rule
    public function __wakeup() {
        throw new \Exception('Cannot unserialize a singleton.');
```

public function manageUsers(string \$action): void {
echo "[Admin Session] Executing User Action: {\$action}\n";
}
}
// --- Execution Example ---
echo "Starting system...\n";
// Pointer 1 tries to access Admin
\$adminSession1 = Admin::getInstance();
\$adminSession1->manageProducts('Update stock for KB-104');
// Pointer 2 tries to access Admin
\$adminSession2 = Admin::getInstance();
\$adminSession2->manageUsers('Ban malicious user account');
// Prove they are the exact same instance in memory
if (\$adminSession1 === \$adminSession2) {
echo "\nResult: Both variables share the same Singleton Admin instance.\n";
}
?>

4.3.5. Benefit

- **Controlled Access to a Sole Instance:** It strictly encapsulates its sole instance, giving the system absolute control over how and when clients can access administrative functions.
- **Reduced Namespace Pollution:** It avoids the use of global variables that can be easily overwritten or corrupted by other parts of the system, protecting sensitive administrative data.
- **Lazy Initialization:** The admin object is only created into the server's memory when it is requested for the first time, conserving system resources if no admin logs in for a period.

4.3.6. Limitations

- **Hidden Dependencies (Global State):** Because the Singleton can be accessed from anywhere in the codebase using `Admin::getInstance()`, it acts like a global variable. This can make the system's architecture tightly coupled and harder to unit test, as parts of the code secretly rely on the Admin state.
- **Concurrency Issues:** In a large e-commerce system with thousands of simultaneous threads, if two threads call `getInstance()` at the exact same millisecond before the instance is created, the system might accidentally create *two* instances. This requires strict thread-locking mechanisms (like the `std::mutex` shown in the code) which can slightly slow down system performance.

5. Conclusion and suggestion

This project shows that we can use known software design patterns to build a modular and scalable e-commerce platform. We used patterns like Strategy and Observer and Singleton in a system. This way the system is easy to understand. Each part is not too connected to the others. This means that important parts like user management and product catalogue and shopping cart and order processing can be changed or updated without affecting the system.

Using these patterns helps to solve problems in e-commerce systems like making checkout easier and keeping track of inventory in real time and managing resources.

This project also shows that it is an idea to separate different parts of the system using a Three-Tier Architecture. This makes the system easier to maintain and scale. It is easier to add new features later. The research we did also shows that it is important to think about how easy the systems to use and to use microservices architecture and to follow standards when building a strong e-commerce platform.

Now we are focusing on the basic parts of the system but in the future, we could add more advanced features like using artificial intelligence to recommend products and making it easier for other sellers to use the platform and automating customer support. Overall, this project provides a base, for building e-commerce solutions that are reliable and can be easily changed and are focused on the user and can adapt to changing business and technology needs.

6. Bibliography/Reference

- Psaromiligkos, Y., Sartzetaki, M., Retalis, S., & Avgeriou, P. (2003). *An approach for usability evaluation of e-commerce sites based on design patterns and heuristics criteria*. In Proceedings of the International Conference on Human-Computer Interaction. University of Groningen.
https://www.researchgate.net/publication/30496977_An_approach_for_usability_evaluation_of_e-commerce_sites_based_on_design_patterns_and_heuristics_criteria
- International Journal of Scientific Development and Research. (2025). *Microservices architecture in e-commerce: Design patterns and scalability strategies*. *IJSDR*, 10(1), b190–b204.
[IJSDR Official PDF](#)
- Wulfert, T., Woroch, R., Strobel, G., Seufert, S., & Möller, F. (2022). *Developing design principles to standardize e-commerce ecosystems: A systematic literature review and multi-case study of boundary resources*. *Electronic Markets*, 32(4), 1813–1842.
<https://doi.org/10.1007/s12525-022-00558-8>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
[Chrome extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.javier8a.com/itc/bd1/articulo.pdf](https://www.javier8a.com/itc/bd1/articulo.pdf)
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall. <https://share.google/AnyejeDgPyn6EI5cI>
- Laudon, K. C., & Traver, C. G. (2023). *E-commerce 2023: Business, technology, society* (18th ed.). Pearson. <https://share.google/25ol4FYymoXuWQ29Q>
- Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley. <https://share.google/WoW7kFO10pZOSYufk>
- Banks, A., & Porcello, E. (2020). *Learning React: Modern patterns for developing React apps* (2nd ed.). O'Reilly Media.
[https://ebooks.karbust.me/Technology/Learning%20React%20-%20Modern%20Patterns%20for%20Developing%20React%20Apps%20-%20Alex%20Banks,%20Eve%20Porcello%20-%20O'Reilly%20Media%20\(2020\).pdf](https://ebooks.karbust.me/Technology/Learning%20React%20-%20Modern%20Patterns%20for%20Developing%20React%20Apps%20-%20Alex%20Banks,%20Eve%20Porcello%20-%20O'Reilly%20Media%20(2020).pdf)